Escola de Camins
Escola Tècnica Superior d'Enginyeria de Camins, Canals i Ports
UPC BARCELONATECH

TREBALL FINAL DE GRAU

Exploring the Use of Machine Learning Techniques in Solid Mechanics Applications

Treball realitzat per:
Juan Águila Martínez

Dirigit per:
Dr. Alejandro Cornejo
Dr. Rubén Zorrilla

Grau en:
Enginyeria Civil

Barcelona, Juliol de 2026

Departament d'Enginyeria Civil i Ambiental

Universitat Politècnica de Catalunya

Escola Tècnica Superior d'Enginyeria de Camins, Canals i Ports de Barcelona

Exploring the Use of Machine Learning Techniques in Solid Mechanics Applications

A Study of Physics Enforcement Strategies Using Synthetic Data

Juan Águila Martínez

Bachelor's degree in
Civil Engineering

Supervisors:
Dr. Alejandro Cornejo
Dr. Rubén Zorrilla

July 2026

Abstract

Constitutive models lie at the heart of modern engineering, providing a mathematical relationship between stress and strain in structural materials that serves as the foundation for any finite element analysis. Traditionally, these models have been phenomenological, but the emergence of new materials challenges this approach. Modern metamaterials and architected materials exhibit highly nonlinear stress–strain relationships that arise, in many cases, from multiscale interactions at the molecular and structural level. In this context, selecting and calibrating a constitutive model frequently becomes a trial-and-error process. Artificial neural networks offer an alternative, learning the constitutive law directly from data as universal function approximators. On their own, however, they guarantee none of the fundamental physical principles, such as thermodynamic consistency or material objectivity, that finite element simulations require to remain well-posed. In response, the field of Physics-Integrated Machine Learning offers strategies falling into two families: Physics-Informed Neural Networks (PINNs), which steer the training process toward physics-satisfying solutions, and Physics-Augmented Neural Networks (PANNs), which constrain the network architecture so that only physically admissible responses can be represented.

This thesis pursues two objectives: first, to present a structured review of the literature in the field of machine learning applied to constitutive modeling; second, to present an experimental exercise that seeks to clarify how the incorporation of physical constraints into the architecture of neural networks affects the accuracy, data efficiency, and extrapolation capabilities of models trained from experimental data. To this end, we implement and compare two architectures: a conventional feedforward neural network (baseline) and a Physics-Augmented Neural Network (PANN). Both networks are trained on synthetic stress–strain data generated from a Neo-Hookean law under plane-strain conditions. On deformation modes included in the training set, both models achieve R^2 above 0.99, with the PANN reducing RMSE by 39% and MAE by 54%. On the other hand, on unseen deformation modes, the baseline's R^2 drops to 0.73, while the PANN maintains 0.99 (a reduction of the generalization gap by 97.7%).

Resumen

Los modelos constitutivos ocupan un lugar central en la ingeniería moderna, estableciendo la relación matemática entre tensión y deformación que sustenta cualquier análisis por elementos finitos. Su desarrollo ha seguido tradicionalmente un enfoque fenomenológico, pero la irrupción de materiales como los metamateriales y los materiales arquitecturados desafía este enfoque. Estos materiales exhiben comportamientos altamente no lineales fruto de interacciones a múltiples escalas, convirtiendo la selección y calibración del modelo en un proceso de ensayo y error.

Las redes neuronales artificiales, en tanto que aproximadores universales de funciones, ofrecen una vía alternativa para aprender la ley constitutiva directamente de datos experimentales. Sin embargo, un enfoque puramente basado en datos no garantiza el cumplimiento de principios físicos fundamentales como la consistencia termodinámica o la objetividad material, ambos indispensables en simulación por elementos finitos. El campo del *Physics-Integrated Machine Learning* aborda este problema desde dos ángulos: las *Physics-Informed Neural Networks* (PINNs), que incorporan la física como restricción durante el entrenamiento, y las *Physics-Augmented Neural Networks* (PANNs), que la integran directamente en la arquitectura para acotar el espacio de soluciones a las físicamente admisibles.

Este trabajo persigue dos objetivos. El primero es ofrecer una revisión estructurada de la literatura sobre aprendizaje automático aplicado a la modelización constitutiva. El segundo es un ejercicio experimental que analiza cómo la incorporación de restricciones físicas en la arquitectura afecta a la precisión, la eficiencia de datos y la capacidad de extrapolación. Para ello, se implementan y comparan dos arquitecturas —una red neuronal *feedforward* convencional (*baseline*) y una PANN— entrenadas con datos sintéticos de tensión–deformación generados a partir de una ley Neo-Hookeana en deformación plana. En los modos de deformación del conjunto de entrenamiento, ambos modelos alcanzan un $R^2 > 0{,}99$, aunque la PANN reduce el RMSE un 39 % y el MAE un 54 %. La diferencia es más pronunciada en modos no vistos: el R^2 del *baseline* cae a 0,73, mientras que la PANN mantiene un 0,99, lo que supone una reducción de la brecha de generalización del 97,7 %.

Resum

Els models constitutius són al cor de l'enginyeria moderna, establint la relació matemàtica entre tensió i deformació que sustenta qualsevol anàlisi per elements finits. El seu desenvolupament ha seguit tradicionalment una aproximació fenomenològica, però la irrupció de materials com els metamaterials i els materials arquitecturats posa aquest plantejament en qüestió. Aquests materials exhibeixen comportaments altament no lineals fruit d'interaccions a múltiples escales, cosa que converteix la selecció i calibratge del model en un procés d'assaig i error.

Les xarxes neuronals artificials, en tant que aproximadors universals de funcions, ofereixen una via alternativa per aprendre la llei constitutiva directament de dades experimentals. Tanmateix, un enfocament purament basat en dades no garanteix el compliment de principis físics fonamentals com la consistència termodinàmica o l'objectivitat material, tots dos indispensables en la simulació per elements finits. El camp del *Physics-Integrated Machine Learning* aborda aquest problema des de dos angles: les *Physics-Informed Neural Networks* (PINNs), que incorporen la física com a restricció durant l'entrenament, i les *Physics-Augmented Neural Networks* (PANNs), que la integren directament en l'arquitectura per delimitar l'espai de solucions a les físicament admissibles.

Aquest treball persegueix dos objectius. El primer és oferir una revisió estructurada de la literatura sobre aprenentatge automàtic aplicat a la modelització constitutiva. El segon és un exercici experimental que analitza com la incorporació de restriccions físiques en l'arquitectura afecta la precisió, l'eficiència de dades i la capacitat d'extrapolació. Amb aquesta finalitat, s'implementen i comparen dues arquitectures —una xarxa neuronal *feedforward* convencional (*baseline*) i una PANN— entrenades amb dades sintètiques de tensió–deformació generades a partir d'una llei Neo-Hookeana en deformació plana. En els modes de deformació del conjunt d'entrenament, tots dos models assoleixen un $R^2 > 0{,}99$, tot i que la PANN redueix el RMSE un 39 % i el MAE un 54 %. La diferència és més pronunciada en modes no vistos: el R^2 del *baseline* cau a 0,73, mentre que la PANN manté un 0,99, la qual cosa suposa una reducció de la bretxa de generalització del 97,7 %.

Acknowledgements

First, I would like to thank my supervisors Alejandro Cornejo and Rubén Zorrilla for their guidance and support throughout this project. I took on the challenge of writing a final degree thesis after nearly fifteen years away from academia. From the choice of topic and the definition of scope to my integration into the existing body of work, their guidance and feedback throughout the thesis have been exceptional.

I also want to thank my wife, Mari Carmen, and the rest of my family for their unconditional support. Writing this work has meant many hours stolen from family life and squeezed between work and everyday responsibilities. They were the ones who encouraged me to take the leap and the ones who gave me the space to see it through.

This project has given me a glimpse into the extraordinary potential of machine learning applied to engineering and physical modelling. I reach the end of this chapter with a renewed enthusiasm for research, convinced that tools like those presented in this thesis can be transformative. I started thinking this would be the closing of a chapter that has spanned nearly twenty years, and I have ended up discovering that it is, perhaps, the beginning of a new one.

Contents

Abstract i

Resumen ii

Resum iii

Acknowledgements iv

1 Introduction 1

1.1 Motivation and Context 1

1.2 Problem Statement 2

1.3 Research Objectives 4

1.4 Scope and Limitations 5

1.5 Document Outline 6

2 Fundamentals of Hyperelasticity and Neural Networks 7

2.1 Introduction 7

2.2 Continuum Mechanics and Hyperelasticity 8

2.2.1 Kinematics of Finite Deformation 8

2.2.2 Principal Stretches 10

2.2.3 Principal Invariants 10

2.2.4 Stress Measures . . . 11
2.2.5 Hyperelasticity . . . 12
2.2.6 Isotropic Hyperelasticity . . . 13
2.3 Neural Networks Fundamentals . . . 14
2.3.1 Artificial Intelligence, Machine Learning, Representation Learning, and Deep Learning . . . 14
2.3.2 Feed-Forward Neural Networks . . . 15
2.3.3 Activation Functions . . . 16
2.3.4 Training Neural Networks . . . 17
2.3.5 Automatic Differentiation and Sobolev Training . . . 18

3 Physics-Enforcement Strategies in ML-Based Constitutive Models 20
3.1 Introduction . . . 20
3.2 First Wave: Early ML-Based Constitutive Models (1990s–2000s) . . . 22
3.3 Second Wave: Deep Learning and Data-Driven Hyperelastic Models (2010s) . . . 24
3.4 Third Wave: Physics-Informed and Mechanically Consistent Models (2020s) . . . 26
3.4.1 Taxonomy of Physics Enforcement Strategies . . . 26
3.4.2 Soft Constraints Through the Loss Term: Physics-Informed Neural Networks . . . 28
3.4.3 Hard Constraints in the Architecture: Physics-Constrained Neural Networks . . . 29
3.4.4 Learning Without Stress Data: Unsupervised Learning and Unsupervised Discovery of Constitutive Models . . . 32
3.5 Architecture-Level Physics Enforcement . . . 34
3.5.1 How to Enforce Physics in Constitutive Models? . . . 34
3.5.2 Thermodynamic Consistency . . . 34

3.5.3 Material Objectivity . . . 37
3.5.4 Material Symmetry . . . 38
3.5.5 Material Stability (Polyconvexity) . . . 38
3.5.6 Growth and Normalization Conditions . . . 40
3.6 Comparative Summary of Strategies and Outlook . . . 41
3.6.1 Comparative Summary . . . 41
3.6.2 Remarks and Interpretation . . . 41
3.6.3 Transition to Implementation . . . 42

4 Methodology and Implementation 43
4.1 Problem Statement . . . 43
4.1.1 Theoretical Setting . . . 43
4.1.2 Objectives . . . 44
4.1.3 Software Organization and Dependencies . . . 45
4.2 Synthetic Data Generation . . . 45
4.2.1 Kratos API Implementation . . . 45
4.2.2 Strain Space Parametrization . . . 46
4.2.3 Admissibility Filtering . . . 48
4.2.4 Dataset Structure . . . 50
4.2.5 Complete Data Generation Pipeline . . . 50
4.3 Data Management in the ML Pipeline . . . 52
4.3.1 Case-Based Splitting . . . 52
4.3.2 Input Normalization . . . 52
4.4 Baseline Model . . . 53
4.5 Physics-Augmented Neural Network (PANN) . . . 54
4.5.1 Definition of the Output (Ensuring Thermodynamic Consistency) . . . 54

4.5.2 Definition of the Input (Ensuring Frame Indifference) 55
4.5.3 Forward Pass Architecture 56
4.5.4 Implementation of the ICNN 56
4.5.5 Correction Terms 58
4.5.6 Stress Differentiation 59
4.6 Training Process 60

5 Numerical Results and Discussion 61
5.1 Introduction 61
5.2 Description of the Dataset 62
5.3 Default Scenario 62
5.3.1 Data Split 62
5.3.2 Interpolation Performance (Seen Scenarios) 64
5.3.3 Extrapolation Performance (Unseen Scenarios) 65
5.3.4 Generalization Gap (Seen vs Unseen Scenarios) 67
5.3.5 Qualitative Stress–Strain Response 68
5.4 Sensitivity to Training Data Composition 70
5.4.1 Alternate Scenario Splits 70
5.4.2 Generalization Gap for Alternate Scenario Splits 71
5.4.3 Per-Component Behavior in Unseen Scenarios 72
5.5 Data Efficiency and Minimum Sample 73
5.5.1 Baseline Performance vs Training Size 73
5.5.2 PANN Performance vs Training Size 75
5.6 Training Dynamics 76
5.6.1 Early Stopping and Convergence 76
5.6.2 Training Time 77
5.7 Discussion 80

5.7.1 On Interpolation . . . 80
5.7.2 On Extrapolation . . . 80
5.7.3 On Data Efficiency . . . 81
5.7.4 On Training Dynamics . . . 82

6 Conclusions and Future Work **84**
6.1 Conclusions . . . 84
6.2 Contributions . . . 85
6.3 Future Work . . . 86

Appendices **87**

A Sustainability Analysis and Ethical Implications **88**
A.1 Sustainability Analysis . . . 88
A.1.1 Environmental Impact . . . 88
A.1.2 Economic Impact . . . 90
A.1.3 Social Impact . . . 91
A.2 Ethical Implications . . . 92
A.3 Relation to the Sustainable Development Goals . . . 92

B Software Organization **93**

C Dependencies **97**

References **99**

List of Figures

2.1 A Venn diagram showing how deep learning is a kind of representation learning, which is in turn a kind of machine learning, which is used for many but not all approaches to AI. Source: Goodfellow, Bengio, and Courville (2016). 15

3.1 A hierarchy of physics-based approaches for machine learning-based constitutive modeling. Generated by the author adapting the taxonomy presented by Linden et al. (2023). 28

3.2 Schematic depiction of the common conditions on elastic potential and stresses: (a) thermodynamic consistency, (b) symmetry of the Cauchy stress, (c) objectivity, (d) material symmetry, (e) polyconvexity, (f) volumetric growth condition, (g) and (h) normalization conditions for energy and stress, as well as (i) non-negativity of energy. Within (e)–(i), solid blue lines denote models accounting for the respective condition, while the dashed red lines mark models which violate it. In (c) and (d), $d\mathbf{X}$, $d\mathbf{X}^*$, $d\mathbf{x}$ and $d\mathbf{x}^*$ denote material line elements for different deformation states, where $d\mathbf{X}^*$ and $d\mathbf{x}^*$ are transformed by a rotation of $d\mathbf{X}$ and $d\mathbf{x}$, respectively. Source: Linden et al. (2023). 36

4.1 Spherical sampling of the admissible strain space in Voigt notation ($\mathbf{E}_{\text{Voigt}} = [E_{11}, E_{22}, \gamma_{12}]$). Each point on the sphere corresponds to a strain state generated via Kratos Multiphysics at a fixed angular position and variable radius, representing increasing deformation magnitude. The radial lines correspond to loading paths at fixed angular coordinates, sweeping from zero to the maximum radius. . . 49

4.2 Example stress–strain curve for a single loading case on the surface of the sphere. 51

4.3 Forward pass of the PANN: Green–Lagrange strain, $\mathbf{E} \rightarrow$ right Cauchy–Green tensor, $\mathbf{C} \rightarrow$ isotropic invariants, $[\mathrm{I}_C, \mathrm{II}_C, \mathrm{III}_C, -2J]$ $\rightarrow$ ICNN energy, ψ_{ICNN}, combined with a reference-state correction, ψ_{ref} (c at reference state) and volumetric penalty, ψ_{vol}, into the total strain energy, ψ_{PANN}, differentiated with respect to $\mathbf{C}$ via automatic differentiation to produce the PK2 stress, $\mathbf{S} = 2\,\partial\psi/\partial\mathbf{C}$. 57

5.1 Stress response along a simple-shear path 69

5.2 Stress response along an equibiaxial path 69

5.3 Stress response along an off-axis path (94.7°) 69

5.4 Stress response along an off-axis path (56.8°) 70

5.5 Generalization gap (R^2 seen minus unseen) by scenario split. The baseline gap varies from 0.049 to 0.255 depending on training diversity; the PANN gap is approximately constant at 0.013–0.015 after the first outlier of 0.105. 72

5.6 Per-component R^2 on unseen data for three representative splits. The baseline's S_{12} is consistently its weakest component; the PANN shows no such asymmetry. 74

5.7 R^2 versus training set size for the baseline (left) and PANN (right). The baseline achieves high seen-scenario R^2 with very little data but its unseen R^2 peaks and then declines. The PANN fails completely below ∼1,300 samples, then rapidly converges to high accuracy on both seen and unseen data. 76

5.8 MSE loss for both models on the "original" split. The baseline (red) converges quickly and early-stops. The PANN (blue) shows a long plateau followed by rapid descent, and is still improving at epoch 150. 77

5.9 PANN validation loss across training epochs for five training set sizes. Larger datasets produce shorter plateau phases and faster convergence. At 10% (262 samples) and 25% (656 samples), the model remains on the plateau throughout training. At 50% (1,312 samples), convergence begins but is not complete by epoch 150. At 75% and 100% (1,968 and 2,625 samples), the model transitions into the rapid-descent phase and approaches convergence, with the full dataset converging earliest (around epoch 80). 78

5.10 Training wall time (solid lines) and epoch count (dotted lines) versus training set size. The baseline early-stops in fewer epochs as data increases; the PANN always runs the full 150 epochs. 79

List of Tables

3.1 A list of requirements for Physics-Augmented Neural Networks. Adapted from Linden et al. (2023). 35

3.2 High-level comparison of the primary strategies for physics enforcement in machine learning-based constitutive models. 41

5.1 Number of Cases and Samples in the dataset. There are 12 named scenarios corresponding to different strain modes. Each one defines a set of cases by sweeping the directional or magnitude component. For each case, we generated augmentations incrementally over 25 steps, resulting in 637 cases and 15,925 samples. 63

5.2 Aggregate interpolation metrics on the seen-scenario test set (875 samples from 7 training loading types). Both models are evaluated on held-out cases not used during training. 64

5.3 Per-component interpolation metrics on the seen-scenario test set (875 samples from 7 training loading types). Both models are evaluated on held-out cases not used during training. 65

5.4 Aggregate extrapolation metrics on the unseen-scenario test set (11,550 samples from 5 unseen loading types). 66

5.5 Per-component extrapolation R^2 on the unseen-scenario test set (11,550 samples from 5 unseen loading types). 66

5.6 Generalization gap, measured as performance in the seen vs unseen test sets, for the different metrics. 67

5.7 Per-component generalization gap, measured as performance in the seen vs unseen test sets, for R^2 and RMSE. 68

5.8 R^2 on seen and unseen scenarios across five training splits of increasing diversity. The gap column reports the difference between seen and unseen R^2. 71

5.9 Per-component R^2 on unseen scenarios for three representative splits. 73

5.10 Baseline performance vs. training set size. Unseen R^2 peaks at 1,312 samples and declines with additional data. 74

5.11 PANN performance vs. training set size. The model fails to converge below approximately 1,300 samples, then transitions sharply to high accuracy. 75

5.12 Training cost for both models across training set sizes. The PANN runs the full 150 epochs in every configuration; the baseline early-stops between 69 and 103 epochs. 77

Chapter 1

Introduction

1.1 Motivation and Context

Over the past three decades, the Finite Element Method (FEM) has become the *gold standard* for complex numerical simulations in civil and structural engineering, and it is used for problems ranging from concrete shell structures to modeling soft tissues in biomechanics (W. K. Liu, Li, and Park 2022). However, the accuracy of any FEM simulation is bounded by the quality of the representation offered by the underlying **constitutive model**. Constitutive models are the backbone of structural engineering, providing a mathematical description of the relationship between stress and strain and determining whether a FEM simulation can reliably predict real structural behavior.

Historically, constitutive modeling has relied on phenomenological models derived from experimental observation of macroscopic behavior and physical intuition. In the domain of hyperelasticity (finite-strain elasticity), we usually rely on models such as Neo-Hookean, Ogden, or Mooney–Rivlin, which are well established for most standard rubber-like materials. However, when applied to highly non-linear materials such as modern metamaterials and architected materials (e.g., honeycomb structures and foam-like structures) or complex biological tissues, selecting and calibrating the "correct" model becomes a manual trial-and-error process that is often prone to user bias. Moreover, when an architected material is treated

as a classical continuum, its effective behaviour may violate the admissibility and stability conditions required of a conventional constitutive model, so that no thermodynamically consistent analytical form exists to be fitted in the first place.

Scientific Machine Learning (SciML) offers a potential alternative to this approach by using neural networks, known as universal function approximators, to learn the constitutive behavior from experimental data. Rather than defining and validating an analytical expression for the constitutive law, the data-driven method bypasses the restriction to fixed mathematical forms by using a high-dimensional learner that fits the training data with high precision.

However, as the saying popularized by Milton Friedman goes, "*There ain't no such thing as a free lunch*", and getting to this future of data-driven models requires the field to solve a relevant flaw: standard neural networks know nothing about physics, and in the absence of the right framework they can violate very fundamental principles such as conservation laws or thermodynamic consistency, causing numerical divergence and instability when used in FEM solvers.

1.2 Problem Statement

The lack of "physical intelligence" we just described represents the central problem in the field of SciML. The application of SciML to continuum mechanics is a very active research field, with about 4,900 papers published in 2025[1], and the use of machine learning-based models in constitutive laws is advancing fast, bringing solid proposals on how to integrate physical constraints into Neural Network architectures to leverage the potential of the data-driven approach inside a physically robust framework.

As we will see in Chapter 2, the first attempts to learn constitutive laws from data date back to 1991 (Ghaboussi, Garrett, and X. Wu 1991) and focused on training a naive Feed-Forward Neural Network (FFNN) with pairs of experimentally obtained stress–strain vectors to map stress directly from strain. This approach

[1]Source: Google Scholar, search query: "Continuum Mechanics" AND "Machine Learning" (accessed April 10, 2026).

is known as "black box" modeling, as it is agnostic to physics and maximizes the network's expressive potential. As mentioned in the previous section, while "black box" models could fit stress–strain pairs when trained with sufficient data, there is no guarantee they respect the fundamental physical principles that frame the problem (e.g., thermodynamics), and they cannot be integrated into numerical solvers due to their lack of stability and consistency, especially when extrapolating outside the training range.

Since these first attempts, the research frontier has moved from pure data-driven regression to what is now known as **"Physics-Integrated" Machine Learning (PIML)**. In PIML, the challenge is not just fitting the dataset but doing so while strictly adhering to the laws of continuum mechanics, including conservation laws, thermodynamic principles, and material frame indifference, as non-negotiable attributes of the model.

Unfortunately, Milton Friedman's quote applies, again, in the opposite direction: adding constraints to the Neural Network architecture so it matches physical constraints reduces the expressiveness of the model (we are limiting the hypothesis space the model can explore), so finding the best strategy to manage this tradeoff between robustness and expressivity is the central theme of PIML.

As a summary, the current landscape of the PIML field distinguishes different strategies in how to enforce physics in the model:

- On one side, **Physics-Informed models** propose penalizing the learning process when the solution violates physics through the loss function.
- On the other side, **Physics-Augmented models** move towards including constraints in the architecture of the neural network that guarantee physical properties by construction and regardless of the training data.

Like any other expanding field of science, the theoretical landscape of PIML is changing rapidly, strategies blend together and become diffuse, and even high-level taxonomies like the one just introduced (Physics-Informed vs Physics-Augmented models) feel inaccurate when you get to a sufficient level of detail.

At the time of writing this bachelor's thesis, there remains a gap in a compara-

tive, fundamental understanding of the different PIML proposals for hyperelastic problems. For an engineer entering the field, it can be unclear whether the added complexity of physics-constrained architecture yields a real benefit over a standard naive network when data is sparse or when the model must extrapolate to unseen loading trajectories.

1.3 Research Objectives

The primary objective of this bachelor's thesis is to implement, evaluate, and compare different Neural Network architectures for isotropic hyperelastic constitutive modeling, with a particular focus on the implications and impact of enforcing physics through architecture. We will use data generated with Kratos Multiphysics (Dadvand, Rossi, and Oñate 2010) to produce a controlled "ground truth" for rigorous validation.

During the document, we plan to address the following research questions:

1. **RQ1:** *What are the state-of-the-art strategies for enforcing physical constraints (objectivity, thermodynamic consistency, etc.) in machine learning-based constitutive models?*

2. **RQ2:** *How does enforcing physical constraints by construction (in the neural network architecture) compare in terms of data efficiency and extrapolation to naive "black box" learning?*

To answer these two questions, we will establish the following objectives:

1. **Discussion:** To describe the historical evolution and state-of-the-art of PIML and present it in a clean, comprehensive framework, so it provides the fundamental knowledge to understand the experiment and results and so others can follow the research line in the future.

2. **Implementation:** To implement two different machine-learning based constitutive models: a baseline FFNN and a Physics-Augmented Neural Network.

3. **Evaluation:** To evaluate these models against a controlled "ground truth". Unlike experimental studies, where the true material behavior is unknown, we will use synthetic data generated with Kratos Multiphysics using a Neo-Hookean model. This will allow for precise error quantification.

4. **Comparison:** To evaluate the performance of the different models in situations of (1) data scarcity (sparse training points) and (2) heavy extrapolation (predictions far from the training domain, unseen deformation modes, such as training on uniaxial tension and pure shear and testing on complex, multiaxial strain trajectories).

1.4 Scope and Limitations

We must ensure the feasibility of this work in the context of a bachelor's thesis (12 ECTS), so we will strictly define the scope and limitations as follows:

- **Material behavior:** All the implementations will be limited to isotropic, hyperelastic materials (path-independent, reversible deformations). For the reader's interest, in the document, we may highlight relevant research lines in the broader field of machine learning applied to continuum mechanics (e.g., applications to materials with memory), but these won't be formally included in the scope.

- **Data:** In this bachelor's thesis, we rely exclusively on synthetic data generated from known analytical laws (Neo-Hookean) using Kratos Multiphysics. This will remove experimental uncertainties (e.g., geometric defects) and allow us to focus on the models' learning capabilities.

- **Development:** We will implement a baseline "physics-agnostic" model and a Physics-Augmented model. Several strategies and architectures, such as Kolmogorov–Arnold Networks, will be reviewed conceptually but kept outside the scope of the implementation.

1.5 Document Outline

The remainder of this bachelor's thesis is organized as follows:

- In Chapter 2, we will provide a theoretical background on continuum mechanics and machine learning, which will make the work self-contained and will allow us to introduce the nomenclature and key terminology we will use in the rest of the document.
- In Chapter 3, we will present a literature review and taxonomy of the field of PIML, with a focus on approaches and strategies for hyperelastic materials. The main mission of this chapter is to categorize the state of the art in PIML for hyperelasticity, providing the necessary background to follow through with the rest of the document and to lay a foundation for future work lines. This chapter directly responds to RQ1.
- In Chapter 4, we will describe the methodology followed in the practical implementation, describing the data generation process, the specific architecture implemented and the coding details (libraries, etc.).
- In Chapter 5, we will present the numerical results, with a comparative analysis of the different approaches in terms of accuracy, stability, "data-eagerness" and ability to extrapolate.
- Finally, in Chapter 6, we will conclude with a summary of findings and trade-offs, outlining future research directions.

Chapter 2

Fundamentals of Hyperelasticity and Neural Networks

2.1 Introduction

The central topic of this project sits at the intersection of two disciplines: continuum mechanics and machine learning. These are two communities using different notations, jargon, and bodies of knowledge. A researcher with an engineering background may know what polyconvexity means but not what an activation function does. Conversely, a researcher with a machine learning background may be fluent in backpropagation but have never heard of the second Piola–Kirchhoff stress. To help others follow this work, we must provide tools to close this gap.

This chapter is an exercise in building a shared vocabulary for the reader and for the ones to follow our work. While we will be rigorous where it matters, this chapter does not aim to be a textbook on either subject. Our goal is a self-contained overview: enough depth to understand why each concept matters for our problem and enough precision to make the notation and terminology unambiguous from this point forward.

2.2 Continuum Mechanics and Hyperelasticity

This section follows the structure of Bonet and Wood (2008), which has been used extensively to prepare the bachelor's thesis. We will also adopt their notation for the rest of the document.

2.2.1 Kinematics of Finite Deformation

Let us start with a snapshot of the material before we stretch, compress, or twist anything. We will call this starting point the **reference configuration**. Mathematically, we can describe this body as a region $\mathcal{B}_0 \subset \mathbb{R}^3$ at time t_0. In this state, every material particle is tagged by its position vector $\mathbf{X}$. This reference configuration will remain the same regardless of how we later deform the body, helping us keep track of "who is who".

Once we apply loads, the body deforms, and each particle $\mathbf{X}$ moves to a new position. Now, the particle is tagged by its *new* position vector, $\mathbf{x}$ (note the lowercase). This end state, called the **current (or spatial) configuration**, is described by the region $\mathcal{B}_t \subset \mathbb{R}^3$ at time t.

The function that maps $\mathbf{X}$ to $\mathbf{x}$ at time t is the motion, written as $\mathbf{x} = \varphi(\mathbf{X}, t)$. Mathematically, we demand this mapping to be smooth, invertible, and orientation-preserving, because material particles cannot teleport, overlap or turn inside out. Now, the central question is how to characterize the transition between the reference and current configurations.

The answer to this question starts with the **deformation gradient**, $\mathbf{F}$, a two-point tensor[1] that can act on an infinitesimal line element, $d\mathbf{X}$, moving it from the reference configuration to the current configuration:

$$d\mathbf{x} = \mathbf{F}\, d\mathbf{X} \tag{2.1}$$

We can view $\mathbf{F}$ as a linear map that encodes stretching, shearing, and rotation.

The determinant of the deformation gradient, known as the **Jacobian**, J, is

[1] A tensor mapping vectors in the reference configuration to the current configuration.

the ratio of a small volume element after deformation to its original volume. As matter cannot pass through itself nor be reduced to zero volume, this scalar must be positive:

$$J = \det \mathbf{F} = \frac{dV}{dV_0} > 0 \tag{2.2}$$

The deformation gradient accounts for both stretching of the material *and* rigid-body rotation. To separate these effects, we use the **polar decomposition theorem**. This theorem states that any non-singular $\mathbf{F}$ can be uniquely split as:

$$\mathbf{F} = \mathbf{RU} = \mathbf{VR} \tag{2.3}$$

Here, $\mathbf{R} \in SO(3)$ is a proper orthogonal rotation tensor ($\mathbf{R}^\mathsf{T}\mathbf{R} = \mathbf{I}$, $\det \mathbf{R} = 1$), and $\mathbf{U}$ and $\mathbf{V}$ are the symmetric, positive-definite **right** and **left stretch tensors**, respectively. The difference is the following: the first ($\mathbf{RU}$) applies a stretch, then a rotation; the second one ($\mathbf{VR}$) applies a rotation and then a stretch. Both decompositions are equivalent for our purposes.

From the stretch tensors, we can build objective strain measures. On one hand, we have the **right Cauchy–Green tensor**, $\mathbf{C}$, defined in the reference configuration, filtering out the rotation:

$$\mathbf{C} = \mathbf{F}^\mathsf{T}\mathbf{F} = (\mathbf{RU})^\mathsf{T}(\mathbf{RU}) = \mathbf{U}^\mathsf{T}\mathbf{R}^\mathsf{T}\mathbf{RU} = \mathbf{U}^2 \tag{2.4}$$

On the other hand, we have the **left Cauchy–Green tensor**, $\mathbf{B}$, defined in the current configuration, filtering out the rotation:

$$\mathbf{B} = \mathbf{F}\mathbf{F}^\mathsf{T} = (\mathbf{VR})(\mathbf{VR})^\mathsf{T} = \mathbf{VR}\mathbf{R}^\mathsf{T}\mathbf{V}^\mathsf{T} = \mathbf{V}^2 \tag{2.5}$$

Both $\mathbf{C}$ and $\mathbf{B}$ are symmetric and positive definite, and share the same eigenvalues (the squared principal stretches λ_i^2). However, their eigenvectors point in different directions: for $\mathbf{C}$, they align with the principal axes in the reference configuration. For $\mathbf{B}$, they align with the principal axes in the current configuration.

2.2.2 Principal Stretches

For isotropic materials like those we will deal with in this bachelor's thesis, the strain energy depends only on "how much" the material stretched, not on its orientation. This "how much" is represented by the **principal stretches**, $\lambda_1, \lambda_2, \lambda_3$, which are the singular values of $\mathbf{F}$.

Principal stretches represent the stretches along the principal directions of the material (the three mutually perpendicular directions in the reference body that remain mutually perpendicular after deformation). Along each of these special directions, the material stretches by a factor λ_i, which represents the pure extension or compression in that axis.

There is a mathematical requirement on λ_i to be strictly positive, $\lambda_i > 0$, that follows directly from $J > 0$ (since $J = \lambda_1\lambda_2\lambda_3$). In Chapter 3 we will see that there exists a generalization using signed singular values, ν_i, which allows individual ν_i to be negative (to account for material inversion), while the product $J = \nu_1\nu_2\nu_3$ remains positive. For simplicity, however, we work with the positive principal stretches.

2.2.3 Principal Invariants

It is valid to write the strain energy as a function of principal stretches. However, there is an alternative that is very convenient: the **principal invariants** of $\mathbf{C}$. These are scalar quantities that do not change under orthogonal transformations of the reference frame. Thus, they satisfy material frame indifference (the energy does not depend on the observer's point of view).

On the other hand, as we will see later, using invariants of $\mathbf{C}$ automatically enforces isotropy.

The three principal invariants are:

- $\mathrm{I}_C = \mathrm{tr}(\mathbf{C}) = \lambda_1^2 + \lambda_2^2 + \lambda_3^2$
- $\mathrm{II}_C = \mathrm{tr}(\mathrm{cof}\,\mathbf{C}) = \lambda_1^2\lambda_2^2 + \lambda_2^2\lambda_3^2 + \lambda_3^2\lambda_1^2$
- $\mathrm{III}_C = \det\mathbf{C} = J^2 = (\lambda_1\lambda_2\lambda_3)^2$

2.2.4 Stress Measures

Stress in finite deformation mechanics admits more than one definition. The force acting on a deformed body can be referred either to the original (undeformed) area or to the current (deformed) area, and each choice gives rise to a different stress measure with its own conjugate strain. Which one to use depends on the problem at hand.

The **Cauchy stress**, $\boldsymbol{\sigma}$, is the most intuitive measure: it is the force per unit deformed area. It is the "true" stress. It is symmetric ($\boldsymbol{\sigma} = \boldsymbol{\sigma}^{\mathsf{T}}$) thanks to the balance of angular momentum. However, $\boldsymbol{\sigma}$ does not conjugate to $\mathbf{F}$ or $\mathbf{C}$, which will make it awkward to use when working in constitutive laws from an energy potential.

The **first Piola–Kirchhoff stress**, $\mathbf{P}$, is a two-point tensor. It relates forces in the current configuration to areas in the reference configuration. It naturally pairs with the deformation gradient. However, it is generally not symmetric, which complicates its use. $\mathbf{P}$ and Cauchy stress relate through the Piola transformation:

$$\mathbf{P} = J\,\boldsymbol{\sigma}\,\mathbf{F}^{-\mathsf{T}} \tag{2.6}$$

The **second Piola–Kirchhoff stress**, $\mathbf{S}$, is obtained by pulling back the force vector entirely into the reference frame:

$$\mathbf{S} = \mathbf{F}^{-1}\mathbf{P} = J\,\mathbf{F}^{-1}\boldsymbol{\sigma}\,\mathbf{F}^{-\mathsf{T}} \tag{2.7}$$

This tensor is symmetric and exists entirely in the reference configuration, the body's initial, undeformed state. It naturally pairs with the Green–Lagrange strain tensor, which measures deformation relative to this initial state, and is defined as:

$$\mathbf{E} = \frac{1}{2}(\mathbf{C} - \mathbf{I}) \tag{2.8}$$

Because of these properties, $\mathbf{S}$ is the natural output of constitutive models that use $\mathbf{C}$. Our neural networks will learn to predict this stress measure.

2.2.5 Hyperelasticity

In *elastic* materials, constitutive behavior is only a function of the current state of deformation. Under such conditions, any stress measure at a particle $\mathbf{X}$ is a function of the current deformation gradient $\mathbf{F}$ associated with that particle.

In the special case when the work done by the stresses during the deformation process is dependent only on the initial state at time $t = 0$ and the final configuration at time t, the behavior of the material is said to be path-independent, and the material is termed *hyperelastic*.

As a consequence of this path-independent behavior, and considering that $\mathbf{P}$ is work-conjugate to the rate of deformation gradient, $\dot{\mathbf{F}}$, we can establish a **stored strain energy function** or **elastic potential** ψ per unit of undeformed volume as the work done by the stress from the initial to the current position:

$$\psi(\mathbf{F}(\mathbf{X}), \mathbf{X}) = \int_{t_0}^{t} \mathbf{P}(\mathbf{F}(\mathbf{X}), \mathbf{X}) : \dot{\mathbf{F}}\, dt\,; \qquad \dot{\psi} = \mathbf{P} : \dot{\mathbf{F}} \tag{2.9}$$

Because a hyperelastic material stores and releases energy without loss, it must comply with the second law of thermodynamics. For an isothermal process, the Clausius–Duhem inequality requires the dissipation per unit reference volume to be non-negative:

$$\mathcal{D} = \mathbf{S} : \dot{\mathbf{E}} - \dot{\psi} \;\geq\; 0. \tag{2.10}$$

Since the stored energy depends only on the current strain, $\psi = \psi(\mathbf{E})$, its rate is $\dot{\psi} = \frac{\partial \psi}{\partial \mathbf{E}} : \dot{\mathbf{E}}$, and the dissipation reads $\mathcal{D} = \left(\mathbf{S} - \partial\psi/\partial\mathbf{E}\right) : \dot{\mathbf{E}}$. An elastic deformation is reversible and dissipates no energy, so $\mathcal{D} = 0$ for any strain rate, which is only possible with:

$$\mathbf{S} = \frac{\partial \psi(\mathbf{E})}{\partial \mathbf{E}} = 2\,\frac{\partial \psi(\mathbf{C})}{\partial \mathbf{C}}, \tag{2.11}$$

with the factor of two following from $\mathbf{E} = \frac{1}{2}(\mathbf{C} - \mathbf{I})$.

In consequence, any model that obtains stress by differentiating a scalar energy ψ satisfies the dissipation inequality automatically.

From this expression, we can derive a definition of a hyperelastic material as:

$$\mathbf{P}(\mathbf{F}(\mathbf{X}), \mathbf{X}) = \frac{\partial \psi(\mathbf{F}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{F}} \tag{2.12}$$

Or, in terms of the second Piola–Kirchhoff stress and the Right Cauchy–Green tensor:

$$\mathbf{S}(\mathbf{C}(\mathbf{X}), \mathbf{X}) = 2\,\frac{\partial \psi(\mathbf{C}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{C}} = \frac{\partial \psi(\mathbf{C}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{E}} \tag{2.13}$$

Since $\mathbf{E} = \frac{1}{2}(\mathbf{C} - \mathbf{I})$ implies $\dfrac{d\mathbf{E}}{d\mathbf{C}} = \frac{1}{2}$.

By this definition, the stress tensor is a gradient field, which implies energy conservation and path-independence, thus being **thermodynamically consistent** by construction (Linden et al. 2023).

2.2.6 Isotropic Hyperelasticity

Isotropy is defined by requiring the constitutive behavior to be identical in any material direction. This implies that the relationship between ψ and $\mathbf{C}$ must be independent of the material axes chosen and, consequently, ψ must be a function of the invariants of $\mathbf{C}$:

$$\psi(\mathbf{C}(\mathbf{X}), \mathbf{X}) = \psi(\mathrm{I}_C,\, \mathrm{II}_C,\, \mathrm{III}_C,\, \mathbf{X}) \tag{2.14}$$

For anisotropic materials, the stored strain energy also depends on how the deformation affects the fibers. In general, to treat these cases, we introduce additional pseudo-invariants into the formulation, such as $\mathrm{IV}_C = \mathbf{a}_0^\mathsf{T}\,\mathbf{C}\,\mathbf{a}_0$, which measures the squared stretch in the fiber direction. Anisotropy falls outside of the scope of this bachelor's thesis (see Section 1.4), so we will focus on the isotropic case from here on.

2.3 Neural Networks Fundamentals

In the previous section, we followed the structure of Bonet and Wood. Here, our primary reference is the Deep Learning manual by Goodfellow, Bengio, and Courville (2016), which provides a comprehensive and rigorous description of neural network fundamentals. To maintain clarity and avoid conflicts with earlier nomenclature, we will explicitly state notation choices and differences as needed to ensure consistency.

2.3.1 Artificial Intelligence, Machine Learning, Representation Learning, and Deep Learning

Today, **artificial intelligence** (AI) is a thriving field, rapidly moving from labs into daily life and becoming more efficient, affordable, and accessible (Maslej et al. 2025). We see intelligent software that automates routine labor, understands speech and images, and supports medical diagnoses and scientific research.

Modern AI effectively addresses problems that cannot be characterized by explicit formal rules. Examples include speech and facial recognition. These systems identify patterns in unstructured data through **machine learning**. Machine learning algorithms use training data to determine the relationship between input variables, often called predictors, and outputs.

The performance of machine learning algorithms depends heavily on the data representation they receive. Choosing the right representation is central because it significantly impacts performance. One solution is to discover not only the mapping from representation to output but also the representation itself. This approach is called **representation learning**. Learned representations usually perform better than hand-designed ones.

Obtaining expressive representations for complex data domains can be nearly as challenging as solving the original problem. For example, in image recognition, object shape varies with viewing angle, distance, and other factors. The challenge is to disentangle these factors. **Deep learning** addresses this by constructing representations hierarchically, building complex concepts from simpler features.

Figure 2.1 illustrates the relationships among the concepts we just introduced.

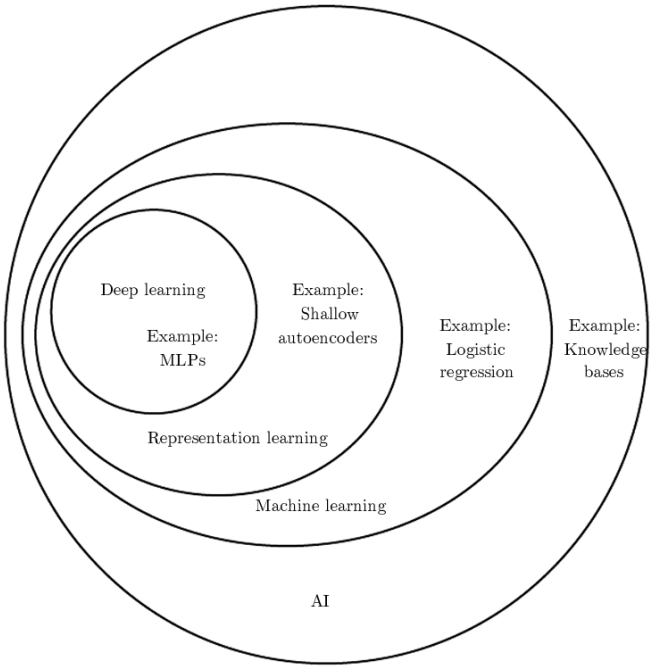

Figure 2.1: A Venn diagram showing how deep learning is a kind of representation learning, which is in turn a kind of machine learning, which is used for many but not all approaches to AI. Source: Goodfellow, Bengio, and Courville (2016).

2.3.2 Feed-Forward Neural Networks

The most popular deep learning algorithm is the previously mentioned Feed-Forward Neural Network (FFNN), which is a mathematical function that maps a set of input values to a set of output values. An FFNN approximates a non-linear mapping $f : \mathbb{R}^{n_{in}} \to \mathbb{R}^{n_{out}}$ through a composition of affine transformations followed by non-linear activation functions, stacked one after another.

Consider a network with L hidden layers (layers that do not correspond to the input or the output). The **input vector** is $\mathbf{h}^{(0)} = \mathbf{x}$ (note that $\mathbf{x}$ is a generic input vector, and not the vector for the reference configuration as in continuum

mechanics). The propagation from layer $l-1$ to layer l follows these two steps:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\,\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \tag{2.15}$$

$$\mathbf{h}^{(l)} = \sigma\Big(\mathbf{z}^{(l)}\Big) \tag{2.16}$$

Here, $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ is the **weight matrix** (determining the strength of connections between neurons), $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ is the **bias vector** (acting as a learnable threshold) and $\sigma(\cdot)$ is the **activation function**, applied element-wise to the pre-activation vector, $\mathbf{z}^{(l)}$.

In FFNNs, each layer takes the output from the previous one, applies a linear transformation ($\mathbf{Wh} + \mathbf{b}$) and then passes the result through a non-linear function (σ). This construction has been demonstrated to be very effective, with a theoretical foundation in the **Universal Approximation Theorem** (Cybenko, 1989; Hornik 1991), which states that an FFNN with a single hidden layer of sufficient width can approximate any continuous function on a compact domain to arbitrary precision. In practice, "sufficient width" can mean a large number of neurons, so deep networks are preferred over shallow ones for their ability to build up hierarchical representations more efficiently.

2.3.3 Activation Functions

The activation function σ is at the center of the expressive power of neural networks. Without an activation function, the composition of affine transformations would collapse into a single affine transformation, resulting in an affine map, no matter how many layers you stack.

As we will see in Chapter 3, the choice of the activation function matters in constitutive modeling. The reason is that, as we saw earlier, the stress is the derivative of the energy potential, and the tangent stiffness is the second derivative, so we need the energy potential to be at least twice continuously differentiable. Taking the Rectified Linear Unit (ReLU) ($f(x) = \max(0, x)$) as an example: this function is piecewise linear. Its first derivative is a step function, and its second derivative is zero everywhere (except at the origin, where it is undefined). A network

built with ReLU would produce piecewise-linear energy, a piecewise-constant stress, and a tangent stiffness that is identically zero. We will see in Chapters 3 and 4 how to overcome this situation and which function we can use. However, it is a good example of how the design space of activation functions in nonlinear mechanics is narrower than in standard deep learning practice.

2.3.4 Training Neural Networks

Training a neural network means adjusting the set of parameters $\boldsymbol{\theta} = \left\{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\right\}_{l=1}^{L}$ (the weights and biases across all layers) so we minimize a scalar loss function $\mathcal{L}(\boldsymbol{\theta})$ that measures how far the network's predictions are from ground truth.

If we use our constitutive modeling problem as an example, the loss measure is defined by the discrepancy between the stress predicted by the network and the stress observed in the experimental data:

$$\mathcal{L}_{\text{MSE}}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \left\|\mathbf{S}_{\text{pred}}(\mathbf{C}_i;\boldsymbol{\theta}) - \mathbf{S}_{\text{exp},i}\right\|^2 \tag{2.17}$$

Physics-Informed approaches can augment this basic MSE with additional terms that penalize violations of the equilibrium equations or enforce boundary conditions. The relative weighting of these terms introduces a design choice that we will revisit in Chapter 3.

The problem we just presented is an optimization problem, that relies on computing the gradient of the loss with respect to the parameters, $\nabla_{\boldsymbol{\theta}}\mathcal{L}$. This is done through the **backpropagation** algorithm, a recursive application of the chain rule that propagates error signals from the output layer backward through the network:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} \left(\mathbf{h}^{(l-1)}\right)^{\mathsf{T}} \tag{2.18}$$

Once we obtain the gradients through backpropagation, parameters are updated via gradient descent:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}}\mathcal{L} \tag{2.19}$$

Where η is the learning rate.

Modern training uses adaptive optimizers like **Adam** (Adaptive Moment Estimation), by Kingma and Ba (2015), which maintain running estimates of the first and second moments of gradients and adjust the effective learning rate for each parameter independently, helping them to navigate the loss landscape in a more effective way.

2.3.5 Automatic Differentiation and Sobolev Training

As in many other engineering domains, symbolic differentiation (taking a mathematical expression and producing its exact derivative using the rules of calculus) is usually not possible in neural networks, given the number of parameters (typically thousands to millions).

Numerical differentiation can help us approximate derivatives through finite differences:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \tag{2.20}$$

This is a popular approach, simple to implement, but suffering from truncation error (if h is too large) and round-off error (if h is too small), and it scales poorly to high-dimensional functions.

Automatic Differentiation (AD) approaches the problem by decomposing the function into a computational graph of elementary operations (additions, multiplications, exponentials, etc.), and applying the chain rule mechanically through the graph. The result is a derivative exact to machine precision, computed at a cost proportional to the cost of evaluating the original function.

In constitutive modeling, AD serves a dual purpose:

- First, it computes $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ for backpropagation (it is the standard use of AD in deep learning).
- Second, it computes the stress tensor $\mathbf{S}$ and the tangent stiffness $\mathbb{C}$ from the

energy potential ψ:

$$\mathbf{S} = 2\,\frac{\partial \psi_{\mathrm{NN}}}{\partial \mathbf{C}} \tag{2.21}$$

$$\mathbb{C} = 2\frac{\partial \mathbf{S}_{\mathrm{NN}}}{\partial \mathbf{C}} \tag{2.22}$$

Therefore, we define a neural network to output a scalar energy ψ_{NN}, but we can train on stress data $\mathbf{S}_{\mathrm{NN}}$ by differentiating through the network at training time. This process is called **Sobolev training** (training on gradients in addition to function values), and it ensures that the learned stress field is the gradient of a potential by construction, not by approximation.

Chapter 3

Physics-Enforcement Strategies in ML-Based Constitutive Models

3.1 Introduction

Machine learning-based constitutive modeling has gone through a very clear generational shift. As we discussed in Chapter 1, neural networks emerged in this domain with the promise of **universal function approximation**. However, this promise comes with serious caveats. Without constraints, black-box models may violate physical fundamentals such as objectivity, fail to meet physical requirements, such as conservation of energy, or become unstable under strain states outside the training regime.

The research community responded by asking two key questions: *how much physics is needed in machine learning-based constitutive models*, and *where it should be embedded (in the loss, in the inputs, or baked into the architecture itself)*. These guiding questions have driven many of the key developments over the past three decades. What began as black-box mappings has matured into a sophisticated field, now defined by hybrid approaches that blend data-driven flexibility with physical interpretability.

In hyperelasticity, the state-of-the-art mixes competing and complementary strategies. Approaches range from supervised models with penalization terms on

one end to physics-based architectures that embed mechanical constraints into the model.

This melting pot of methodologies is the result of different research teams seeking the **right balance between the expressive power of neural networks and the interpretability of traditional constitutive laws**. As in most engineering fields, there is no single "right" solution to this problem, and the modeler must decide where to draw the line between flexibility and control. This decision requires a deep understanding of the different methods. The intent of Chapter 3 is to provide clarity for future researchers by building a framework they can use to make these kinds of decisions.

To give clarity, we have organized this progression into three waves following the historical development of the discipline, each of them largely a reaction to the limitations of the previous one:

1. The **first wave,** in the 1990s and early 2000s, led by Ghaboussi and co-workers (Ghaboussi, Garrett, and X. Wu 1991), demonstrated that FFNNs could learn stress–strain behavior from experimental data. Their work was still constrained by limited architectural, data, and computational resources, but clearly paved the way for later research.

2. The **second wave**, in the 2010s saw deep learning and the broader data-driven movement emerging. Wider tool availability (computational resources, libraries, frameworks) enabled bigger and more expressive networks and larger, often simulated, datasets. Yet, this wave did not address most concerns about the black-box nature of machine learning models.

3. By the 2020s, the conversation had shifted to interpretability and robustness. The ability of the models to generalize under data scarcity became as important as fitting accuracy. In this **third wave**, researchers embedded physical priors into network architectures. Physics-Informed neural networks (PINNs) (Raissi, Perdikaris, and Karniadakis 2019), and constrained architectures like Input Convex Neural Networks (ICNNs) (Amos, Xu, and Kolter 2017), and Physics-Augmented Neural Networks (PANNs) (Linden et al. 2023), enabled the enforcement of consistency, objectivity, and stability by design.

The following sections explore this evolution in detail, highlighting conceptual turning points, technical contributions, and open questions that continue to shape the field.

3.2 First Wave: Early ML-Based Constitutive Models (1990s–2000s)

In 1991, Ghaboussi, Garrett, and X. Wu (1991) introduced "Knowledge-Based Modeling of Material Behavior with Neural Networks". In this work, they **trained a shallow feed-forward neural network (FFNN) on experimental stress–strain data from concrete under biaxial loading**. The main idea was that materials could be treated as unknown input–output maps. If a network could see enough examples of strain inputs and stress outputs, it could approximate the constitutive mapping:

$$\mathcal{M} : \mathbb{R}^6 \to \mathbb{R}^6 \tag{3.1}$$

where the input and output spaces represent the six independent components of the symmetric strain and stress tensors in Voigt notation.

This study showed that FFNNs could, in principle, capture complex material behavior directly from data, requiring only one essential physical idea: stress depends on the current and past strains (because concrete is path-dependent). The network worked as a black-box function approximator, mapping strain inputs to stress outputs without a predetermined analytical form. As a proof of concept, this work highlighted the flexibility of FFNNs for modeling nonlinear material response. It also pointed to challenges discussed in Chapter 1. Later work identified that such black-box approximations often struggled to generalize, and sometimes violated basic physical principles, such as objectivity or thermodynamic consistency. These methods also needed large datasets to approach physically plausible responses.

Researchers in the late 1990s extended Ghaboussi's approach to improve data efficiency. For example, Ghaboussi, Pecknold, et al. (1998) **introduced an "autoprogressive algorithm"** to address data requirements. The network used in this work was a multi-stage network trained incrementally (or "autoprogressively"),

enabling it to learn complex behaviors from small datasets by gradually expanding its hidden layers.

Other early neural network applications in constitutive modeling included using FFNNs to identify failure criteria for anisotropic materials (Theocaris and Panagiotopoulos 1995). Furukawa and Yagawa (1998) developed a constitutive model for viscoplasticity, training a neural network to reproduce the rate-dependent stress response, treating the FFNNs as a constitutive subroutine within an FE solver. This work showed that FFNNs could, in principle, handle path-dependent behavior by taking the strain history as input.

By the mid-2000s, Ghaboussi's team and others also addressed practical implementation issues. For example, Jung and Ghaboussi (2006) **implemented a rate-dependent FFNN model for viscoelasticity in Abaqus**. They trained an FFNN to replicate a standard viscoelastic solid model, and then used it in finite-element simulations of time-dependent deformation. In these simulations, the network provided stress updates and a consistent tangent stiffness matrix at each step.

An early example of data-driven hyperelasticity methods was provided by Shen et al. (2005). They presented **a neural-network-based hyperelastic model for rubber V-belt materials**. Their network was trained to approximate the material's strain-energy function rather than the stress directly. By using strain tensor invariants as inputs, the model satisfied frame indifference (objectivity). It produced a scalar energy output, from which stress was obtained by differentiation. This was an early instance of embedding hyperelastic physics into neural network model design and, as we will see in the coming sections, a clear foundational step toward PANNs.

There are two takeaways from this period. First, that **FFNNs could approximate nonlinear stress–strain relationships that would otherwise need complex definitions and many material parameters**. Second, that this kind of models have large data demands, no guaranteed physical constraints, and a black-box nature that prevented interpretability of the generated models.

In addition, at the time, computational and software resources were limited,

and even training a shallow network was a serious task. For this reason, early studies kept networks small and usually focused on two-dimensional stress states, particular materials, and specific loading scenarios.

3.3 Second Wave: Deep Learning and Data-Driven Hyperelastic Models (2010s)

The 2010s reshaped deep learning. Major advances occurred in computer vision, speech recognition, machine translation, and other perception-related tasks (Dean 2020). During this decade, GPUs became widely used in deep learning, and machine learning frameworks such as TensorFlow (Abadi et al. 2016) and PyTorch (Paszke et al. 2019) were released.

After a relative lull in the mid-2000s, following the rise of deep learning, interest in data-driven constitutive models grew, and researchers used deeper neural networks and new frameworks to model material behavior. During the 2010s, deep neural networks (DNNs) with multiple hidden layers began to replace the shallow networks of earlier decades. Increasing the size of the hidden block improves the network's expressivity, allowing DNN-based constitutive models to represent more complex material responses, including anisotropy (Linka et al. 2021).

Deep learning models are generally data-hungry and tend to underperform in low-data domains. For this reason, a key trend in this decade was the generation of **training data from high-fidelity simulations, such as finite-element (FE) simulations or molecular dynamics simulations**. These simulations were used to supplement or replace experimental data. For example, Chung, Im, and Cho (2021) used molecular dynamics (MD) to simulate polymer networks under various deformations and then trained a DNN to reproduce the resulting stress–strain behavior. In this work, the DNN was trained on a broad range of MD-generated scenarios (uniaxial, biaxial, pure volumetric, and other deformation modes) to capture the complex nonlinearity of a polymer's hyperelastic response. This approach is a good example of a multiscale method that links atomistic and continuum modeling. By learning from MD simulation results, fine-scale physics

informed a continuum constitutive law via machine learning.

Another advance was **specialized network architectures for materials modeling**. In the broader landscape, the deep learning community developed architectures such as convolutional networks for vision, recurrent networks for sequential data, and transformers for speech and language. In constitutive modeling, researchers created architectures tailored for physical modeling. A notable example is the Deep Material Network (DMN) introduced by Z. Liu, C. T. Wu, and Koishi (2019). DMN replaces fully connected architectures with a hierarchy of sub-networks following homogenization theory in composites. Each building block represents a simple material mixture, so the whole network can approximate the behavior of complex heterogeneous materials. The lesson is that **adding domain knowledge, such as the hierarchical nature of composites, can improve data efficiency and reliability in ANNs**.

In parallel, transfer learning became popular during the 2010s. This technique allows a model trained on one task to be adapted to a new, related task. Transfer learning also found its place in constitutive modeling through weight initialization, using pretrained model parameters as starting values.

Despite these advances, concerns about black-box behavior remained. DNNs can fit training data well, but without constraints, they may not respect physical laws, and they can behave unpredictably outside the training regime. In addition, deeper architectures made interpretability harder than with earlier networks. This made it harder to connect predictions to known mechanisms, such as initial stiffness or volumetric response.

The takeaway from this period was clear: **although purely data-driven constitutive models are powerful, they must be grounded in physical principles**. This realization led to the rise of physics-informed and hybrid methods, combining the flexibility of machine learning with the robustness of mechanics-based models.

3.4 Third Wave: Physics-Informed and Mechanically Consistent Models (2020s)

3.4.1 Taxonomy of Physics Enforcement Strategies

Since the early 2020s, there has been a consistent effort to add physical awareness and constraints into machine-learned constitutive models. These models aim not only to fit the data but also to comply with known physical laws and principles *by design*. However, the field has not settled on *how to do it*. Embedding physics into the learning process (whether through the loss, the architecture, or the input representation) always comes at some cost to the network's expressiveness, and there is no established methodology for managing the tradeoff.

Linka et al. (2021) proposed a general framework for **Constitutive Artificial Neural Networks (CANN),** including, as a notable feature, the **addition of terms to express relevant physical constraints**. As with previous approaches, the network was trained on stress–strain pairs, but in this case, the authors simultaneously incorporated theoretical knowledge (such as symmetry, known small-strain limits, or incompressibility conditions) by adding terms to the loss function or by architectural choices. In this work, the authors showed that blending the data-driven approach with physics-based regularization improved the FFNN's predictive performance.

By 2022, most proposals in the field of machine learning-based constitutive modeling incorporated some form of physics guidance, whether through invariant-based inputs, constrained architectures (hard constraints), adapting the loss function (soft constraints) or hybrid training strategies. These physics-based proposals differ, fundamentally, in the location and method of physical enforcement. Following recent literature (Linden et al. 2023), **we categorize these proposals by the level at which enforcement is applied** using this scheme:

1. **Supervised learning with stress–strain** data trains neural networks to map strains (or strain histories) to stresses. Here, physics can be enforced during training through the loss function or by designing the network architecture

to obey physical constraints.

   (a) **Loss-level enforcement** adds a penalty term for violations directly to the loss function. The network is thus encouraged to make physically consistent predictions, but the FFNN architecture remains unconstrained.

   (b) **Architecture-level enforcement** builds physical laws *directly* into the FFNN structure, ensuring constraints are met on every forward pass. This hard-constraint approach includes developments from Input Convex Neural Networks (ICNNs) to Physics-Augmented Neural Networks (PANNs), as well as recent models like Convex Signed Singular Value Neural Networks (CSSV-NNs) and Input-Convex Kolmogorov–Arnold Networks (ICKANs).

2. **Indirect or unsupervised Learning:** Often, we cannot observe the true stress–strain curve at all material points, especially in experiments. Instead, we might have indirect measurements, such as full-field displacements (e.g., from digital image correlation) and resultant forces. Unsupervised (physics-informed) learning uses such data, treating the task as a reverse-engineering problem. The key idea is to embed an FFNN into an FE simulation and optimize its parameters to match simulated outcomes with experimental observations. With this approach, we train the network without explicit stress labels, using physical laws (e.g., equilibrium and compatibility) as a guide for learning.

Though the previous taxonomy shows different strategies as independent methods, this is somewhat artificial. These classifications are for clarity only and are intended to help other researchers navigate the field. In practice, the methodologies are interconnected, blended, and built on each other's ideas.

Figure 3.1 shows the representation of the different approaches we just described and introduces the following subsections.

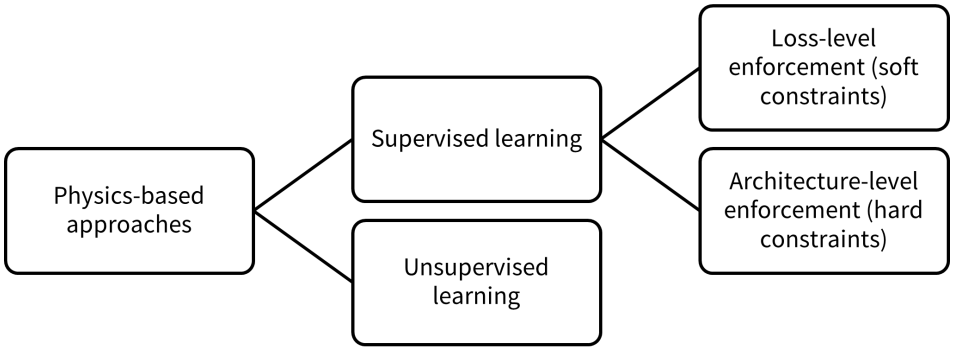

Figure 3.1: A hierarchy of physics-based approaches for machine learning-based constitutive modeling. Generated by the author adapting the taxonomy presented by Linden et al. (2023).

3.4.2 Soft Constraints Through the Loss Term: Physics-Informed Neural Networks

Physics-informed Neural Networks (PINNs), described by Raissi, Perdikaris, and Karniadakis (2019), have been used in mechanics by incorporating physics knowledge as a physics-based loss term during training. This loss term discourages violations of physics, so the loss function is structured accordingly:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \, \mathcal{L}_{\text{physics}} \tag{3.2}$$

Specifically, the physical loss term $\mathcal{L}_{\text{physics}}$ penalizes violations of the mathematical equations that describe how a physical system behaves, such as equilibrium (balance of forces) or thermodynamic (energy conservation) constraints. In this way, the mechanism connects domain knowledge directly with model optimization.

This approach does not guarantee convexity *by construction*; rather, it achieves it *in practice* by penalizing predictions outside the physically admissible domain. Such predictions incur higher losses during training, so the optimization process tends to avoid them and pushes the solution towards physically plausible states.

Consequently, soft constraints alone are not sufficient for safety-critical mechanical modeling. For example, a model might satisfy thermodynamics "on average" (low loss), but violate it locally in specific regions of the state space (Fuhg and

Bouklas 2022). In non-linear FE, a single integration point violating stability can cause the entire simulation to crash. Recognizing these limitations motivated the next generation of research, focused on physical consistency and interpretability.

3.4.3 Hard Constraints in the Architecture: Physics-Constrained Neural Networks

Architecture-level strategies are the most rigorous way to enforce physics: by designing the FFNN architecture so the output space is mathematically restricted to physically admissible functions, we guarantee satisfaction of physical laws *by design*. This remains true regardless of the training data or the optimization process, so these methods can be safely embedded into FE simulations (the physical constraints satisfied in every example, not "on average," as in the case of loss-based approaches).

We also include in this category approaches working directly with the original representation (inputs), such as using invariants as inputs to ensure the FFNN's predictions are not affected by arbitrary rotations of the reference frame (Shen et al. 2005).

Thermodynamic consistency is arguably the most discussed topic in this space. As we saw in Chapter 2, hyperelasticity introduces a scalar strain energy function ψ from which stress follows by differentiation. For the resulting boundary-value problem to be well-posed, ψ must be a convex function of the strain tensor (small strain) or a polyconvex function of the deformation gradient $\mathbf{F}$ (finite strain) (Ball 1977). Standard FFNNs do not guarantee this, so several groups have proposed architectures that enforce convexity in the network's output with respect to its inputs.

Linden et al. (2023) tackled the problem using input-convex neural network (ICNN) architectures, originally proposed in the ML literature by Amos, Xu, and Kolter (2017). In this approach, the authors use non-negative weights and convex, monotonic activation functions (e.g., softplus). This meets the mathematical requirements of a polyconvex energy function, that is, $\psi(\mathbf{F}) = g(\mathbf{F}, \operatorname{cof}\mathbf{F}, \det\mathbf{F})$. Any output strain–energy function is therefore polyconvex by construction, ensuring

material stability. The tradeoff is a slightly less flexible network due to weight constraints, but with guaranteed stability. This is the approach we adopt in our implementation, and we will expand on it through Chapter 4.

Thakolkaran, Guo, et al. (2025) proposed another approach to enforce polyconvexity. They used Kolmogorov–Arnold Networks (KANs) as Constitutive Artificial Networks (CANs) (arriving at the suggestive title "*Can KAN CANs?*"). The Kolmogorov–Arnold representation states that any multivariate continuous function can be expressed as a sum of one-dimensional functions. Based on this, they developed **Input-Convex Kolmogorov–Arnold Networks (ICKANs)** for hyperelasticity. The ICKAN decomposes the strain-energy function into a sum of one-dimensional spline-based sub-networks, each of which is forced to be convex. This architecture has two main advantages over dense ICNNs. First, by using spline-based activations, it achieves good expressivity with fewer parameters, yielding very compact models. Second, the structure makes the model highly interpretable as a network representation of a convex sum of univariate functions. The authors showed that one can perform **symbolic regression on the trained ICKAN** to extract an explicit analytical form of the learned constitutive law, allowing researchers to extract a human-readable expression that can be shared and verified.

In a similar vein, Chen and Guilleminot (2022) proposed an alternative "rectification" strategy. In this work, the authors trained a black-box (unconstrained) FFNN to fit experimental data, and then post-processed (hence "rectified") the FFNN by reassembling it into a new form such that it was mathematically guaranteed to be polyconvex. With this approach, the network can be fully expressive in fitting the data, relegating the convexity requirements to a posterior projection onto a convex function. As a result, they obtained an admissible hyperelastic model that fits the data well and, by construction, satisfied the material stability condition.

There is also a divide in the literature about the right inputs for these convex architectures. The standard method, used by Linden et al. (ICNNs), uses the invariants of the right Cauchy–Green tensor (I_C, II_C, III_C) as network inputs. This guarantees frame indifference. However, Geuken et al. (2025) argue this is

only a sufficient, not a necessary, condition for polyconvexity. The issue lies in the functional limitations of invariant-based ICNNs. Requiring that the strain-energy function is convex and non-decreasing in I_C and II_C, and convex in III_C, restricts which functions can be represented. Geuken et al. note that some valid polyconvex energy functions cannot be captured by this architecture. For example, terms that are linear in the principal stretches (such as in the Ogden model) cannot be represented as convex functions of I_C and II_C because the mapping from invariants to stretches involves square roots, which do not preserve convexity. As a result, invariant-based ICNNs cannot span the space of stable isotropic hyperelastic materials. To address this, Geuken et al. developed Convex Signed Singular Value Neural Networks (CSSV-NNs). Instead of invariants, CSSV-NNs use the signed singular values ν_i of the deformation gradient $\mathbf{F}$. These signed singular values generalize the principal stretches by retaining orientation information. This ensures the mapping is valid for all deformations. CSSV-NNs are designed to overcome the limitations of invariant-based ICNNs, offering a more complete modeling solution, though requiring singular value decomposition as a preprocessing step, which is more computationally demanding than computing invariants.

While convexity remains the focus for hard-constrained machine learning-based constitutive models, researchers have also introduced other physical constraints, such as penalization terms to enforce normalization conditions at the loss level during training. This approach merges soft constraints with the stricter architecture-level strategy discussed above.

Bringing these ideas together and enforcing all these conditions at the architectural level will be the focus of the implementation exercise in this work. Section 3.5 will detail how each requirement can be mathematically implemented in a physics-constrained neural network.

3.4.4 Learning Without Stress Data: Unsupervised Learning and Unsupervised Discovery of Constitutive Models

Beyond enforcement in the architecture to satisfy physical constraints, machine learning-based constitutive models are becoming more interpretable. For hyperelasticity, this trend has two directions: **unsupervised model discovery** and **FFNNs with interpretable or minimalistic expressions**.

The use of unsupervised machine learning methods for constitutive modeling is characterized by the kind of inputs required in the learning process. In supervised training, stress labels, which are often hard to obtain, are needed for each deformation sample. Unsupervised methods learn without direct stress data.

A key example is the EUCLID framework (Efficient Unsupervised Constitutive Law Identification and Discovery) by Flaschel, Kumar, and De Lorenzis (2021). It trains a sparse regression model to identify a suitable strain–energy function from a candidate library instead of learning a stress–strain map. The neural network is embedded in an FE simulation, where its predictions compute internal forces compared against measured forces or displacements, forming a physics-based loss.

The process starts with a large library of invariant-based energy terms (e.g., $(\mathrm{I}_C - 3)^n$). An L_1-regularized regression model (Lasso-like) selects a small subset of those terms that explain the data, resulting in a model interpretable as a combination of Neo-Hookean and polynomial terms that matches experimentally accessible data (displacements and forces)[1].

A notable application is Thakolkaran, Joshi, et al. (2022), who introduced NN-EUCLID, a neural network-based implementation of EUCLID for hyperelasticity. Training data includes full-field displacement maps and resultant forces[2]. The loss function is built on the principle of virtual work and conservation of momentum, forcing the network to satisfy equilibrium at every material point. NN-EUCLID also

[1] It is applicable wherever full-field measurements are possible, such as with 3D Digital Image Correlation.

[2] Following previous footnote, these are quantities that can be measured via DIC and load cells.

incorporated several constraints in the network architecture, including convexity, ensuring the learned strain-energy function was convex in the strain tensor and stable. This work demonstrated that unsupervised physics-informed networks could learn complex, multiaxial constitutive laws from accessible experimental data (displacement fields and reaction forces).

Bourdyot et al. (2025) used micro-CT imaging and 3D digital volume correlation to gather full-field strain data on a polymer under load. They trained a neural network hyperelastic model directly from the 3D experimental data. Using a PANN (with polyconvexity enforced as described earlier), they trained an unsupervised model requiring the network's predictions to satisfy force equilibrium across the sample without direct stress labels.

With symbolic regression, the output is an equation a researcher can inspect, validate, and potentially trust or generalize. Symbolic regression can be computationally intensive and requires careful setup of the search space (choice of function primitives, inclusion of invariant terms, etc.). Nonetheless, the emergence of "*automated model discovery*" tools shows promise for closing the loop between black-box learning and traditional theory.

Another approach to interpretability is **post-hoc analysis of trained networks**. Koeppe et al. (2022) introduced the notion of **Physics-Explaining Neural Networks, in which**, after training a network on mechanical data, techniques such as principal component analysis (PCA) were applied to the network's internal neuron activations to determine whether the network had learned features corresponding to known mechanical quantities. In case studies, they found that certain neurons' behaviors could be correlated with fundamental functions (e.g., one neuron might effectively represent the first invariant I_C, another the volumetric J).

There are also attempts to extract feature importances (e.g., which strain invariant influences the response) from trained models to gain insights. These directions fall outside of what we cover in this bachelor's thesis, but are relevant because they reflect a broader trend: as machine learning matures in constitutive modeling, the community increasingly values *interpretability* alongside pure predictive accuracy.

3.5 Architecture-Level Physics Enforcement

3.5.1 How to Enforce Physics in Constitutive Models?

As mentioned in Section 3.4.1, Linden et al. (2023) presented a comprehensive "guide to enforcing physics" in neural-network hyperelastic models (Table 3.1 and Figure 3.2) and proposed the term **Physics-Augmented Neural Networks (PANNs)** for the family of neural network-based models that satisfy all these requirements exactly. PANNs provide a reference framework that specifies the features a physics-constrained network must include to meet the full set of hyperelasticity requirements.

In the following subsections, we introduce the mathematical formulation of the different requirements. We will use these requirements in Chapter 4 to implement the PANN within an experimental framework.

3.5.2 Thermodynamic Consistency

As we described in Chapter 2, hyperelastic materials establish a strain energy function ψ. We can enforce thermodynamic consistency by designing the network to output a scalar hyperelastic potential ψ rather than stress directly, and then obtaining the stress tensor by differentiating ψ with respect to the deformation measure. We can use the First Piola–Kirchhoff stress $\mathbf{P}$ calculated as the derivative of ψ with respect to the deformation gradient $\mathbf{F}$:

$$\mathbf{P}(\mathbf{F}(\mathbf{X}), \mathbf{X}) = \frac{\partial \psi(\mathbf{F}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{F}} \tag{3.3}$$

Because the stress is computed as the gradient of a potential, it is thermodynamically consistent by construction.

Alternatively, we can use the Second Piola–Kirchhoff stress $\mathbf{S}$ calculated as the derivative of the energy potential ψ with respect to the right Cauchy–Green tensor

Table 3.1: A list of requirements for Physics-Augmented Neural Networks. Adapted from Linden et al. (2023).

	Requirement	Description
a	Thermodynamic consistency	The model must obey the second law of thermodynamics, so it cannot predict unphysical energy creation or dissipation in purely elastic behavior.
b	Symmetry of stress tensor	The Cauchy stress must be symmetric (no net internal torque), which enforces local angular momentum balance.
c	Objectivity	The predicted material response must not change if the whole body is rotated or translated rigidly (frame invariance).
d	Material symmetry	The response must respect the material's intrinsic symmetries (e.g., isotropic materials behave the same in all directions).
e	Polyconvexity	The strain-energy function must have a mathematically stable form that helps ensure existence of solutions and avoids pathological deformations.
f	Growth condition	The energy/stress must grow sufficiently for large deformations so the model remains stable and prevents unrealistic unlimited stretching at finite cost.
g	Normalization condition energy	The strain energy is set to a reference value (typically zero) in the undeformed configuration.
h	Normalization condition stress	The stress is set to zero in the undeformed, unloaded reference configuration.
i	Non-negativity of strain energy	The strain energy must never be negative, so the reference state is not energetically "worse" than a deformed state.

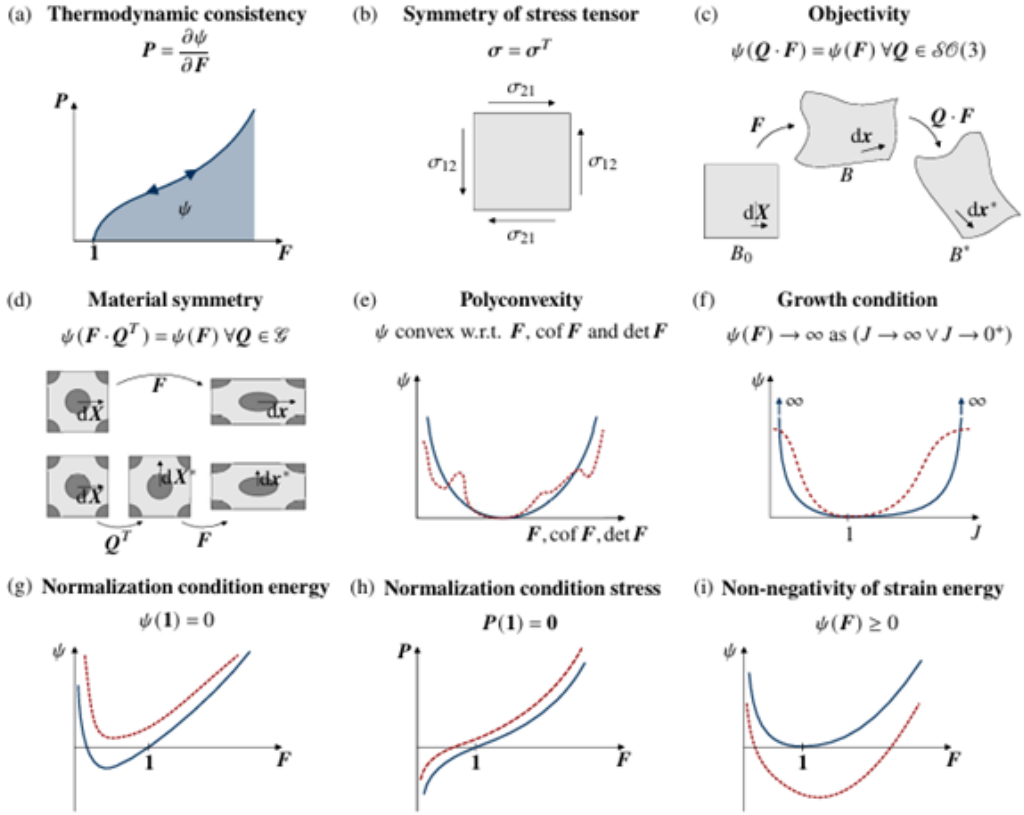

Figure 3.2: Schematic depiction of the common conditions on elastic potential and stresses: (a) thermodynamic consistency, (b) symmetry of the Cauchy stress, (c) objectivity, (d) material symmetry, (e) polyconvexity, (f) volumetric growth condition, (g) and (h) normalization conditions for energy and stress, as well as (i) non-negativity of energy. Within (e)–(i), solid blue lines denote models accounting for the respective condition, while the dashed red lines mark models which violate it. In (c) and (d), $d\mathbf{X}$, $d\mathbf{X}^*$, $d\mathbf{x}$ and $d\mathbf{x}^*$ denote material line elements for different deformation states, where $d\mathbf{X}^*$ and $d\mathbf{x}^*$ are transformed by a rotation of $d\mathbf{X}$ and $d\mathbf{x}$, respectively. Source: Linden et al. (2023).

$\mathbf{C}$ or the Green–Lagrange strain tensor $\mathbf{E}$:

$$\mathbf{S}(\mathbf{C}(\mathbf{X}), \mathbf{X}) = 2\,\frac{\partial \psi(\mathbf{C}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{C}} = \frac{\partial \psi(\mathbf{C}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{E}} \tag{3.4}$$

By this definition, the stress tensor is a gradient field, which implies energy conservation and path-independence, and thus is thermodynamically consistent by construction.

3.5.3 Material Objectivity

A key requirement for any constitutive model is **material frame-indifference (material objectivity)**. It states that the material's response is independent of the observer. In hyperelastic materials, the direct translation is that the strain energy density must remain invariant under superposed rigid body rotations, that is, $\psi(\mathbf{F}) = \psi(\mathbf{QF})$ for any deformation gradient $\mathbf{F}$ and any proper orthogonal rotation tensor $\mathbf{Q}$.

Standard feed-forward neural networks trained on the components of $\mathbf{F}$ as input generally violate this condition, unless trained on massive, rotation-augmented datasets, which is a computationally inefficient solution.

To enforce objectivity by construction, **modern architectures use invariants as inputs**. Rather than processing the deformation gradient directly, the network instead maps the invariants of the right Cauchy–Green deformation tensor $\mathbf{C}$ to the strain energy.

For isotropic hyperelastic materials, the standard set of invariants was introduced in Section 2.2.3:

- $\mathrm{I}_C = \mathrm{tr}(\mathbf{C}) = \lambda_1^2 + \lambda_2^2 + \lambda_3^2$
- $\mathrm{II}_C = \mathrm{tr}(\mathrm{cof}\,\mathbf{C}) = \lambda_1^2\lambda_2^2 + \lambda_2^2\lambda_3^2 + \lambda_3^2\lambda_1^2$
- $\mathrm{III}_C = \det \mathbf{C} = J^2 = (\lambda_1\lambda_2\lambda_3)^2$

By expressing the network as a function of the invariants, $NN\colon (\mathrm{I}_C, \mathrm{II}_C, \mathrm{III}_C) \to \psi$, we ensure frame indifference, as the invariants remain unchanged under rotation of

the reference frame.

3.5.4 Material Symmetry

Choosing the right inputs/outputs not only satisfies thermodynamic consistency and objectivity, but also material symmetry. To reflect material isotropy, the input space is extended with pseudo-invariants derived from structural tensors that represent preferred material directions.

For a transversely isotropic material defined by a unit vector $\mathbf{a}_0$, the input set is expanded to include pseudo-invariants such as

$$\mathrm{IV}_C = \mathbf{a}_0^\mathsf{T}\,\mathbf{C}\,\mathbf{a}_0 \tag{3.5}$$

$$\mathrm{V}_C = \mathbf{a}_0^\mathsf{T}\,\mathbf{C}^2\,\mathbf{a}_0 \tag{3.6}$$

Using these invariants as inputs ensures the model respects the material's symmetry, which is difficult for standard neural networks to learn from data. This architectural bias improves data efficiency by embedding the symmetry.

3.5.5 Material Stability (Polyconvexity)

As mentioned in Section 3.4.3, the biggest challenge in machine-based constitutive models is ensuring the numerical stability of the inferred function ψ to ensure that the solution is well-posed for FE simulations. This stability depends critically on the strain energy function's convexity.

Convexity and Polyconvexity. In linear elasticity, energy must be convex in strain. Nonlinear elasticity cannot require convexity with respect to $\mathbf{F}$ because convexity in $\mathbf{F}$ is incompatible with objectivity.

To resolve this, Ball (1977) introduced the idea of Polyconvexity. A function G is polyconvex if it can be written as a convex function of $\mathbf{F}$ and its minors:

$$\psi(\mathbf{F}) = G(\mathbf{F},\ \mathrm{cof}\,\mathbf{F},\ \det\mathbf{F}) \tag{3.7}$$

Here, G is convex with respect to its arguments.

Polyconvexity means the energy is convex with respect to changes in length, area, and volume ($\mathbf{F}$ controls line, $\operatorname{cof}\mathbf{F}$ area, and $\det\mathbf{F}$ volume elements). In practice, it guarantees the existence of energy minimizers, so FE simulations are well-posed and stable.

For ensuring polyconvexity of the potential predicted by the FFNN, it is necessary to use polyconvex invariants of the symmetry group under consideration and, additionally, adapt the FFNN architecture in a certain way. This requirement led to the adoption and adaptation of Input Convex Neural Networks (ICNNs), and later, they were tailored to address the polyconvexity requirement in computational mechanics.

The ICNN Formulation. An ICNN ensures the convexity of the output scalar (the strain energy ψ) with respect to its inputs. It does this by constraining the neural network weights and activation functions.

Going back to the definition provided in Section 2.3.2, a typical layer in an ICNN is defined as:

$$z_{l+1} = \sigma\Big(W_l^{(z)}\, z_l + W_l^{(y)}\, y + b_l\Big) \tag{3.8}$$

Where y is the original input (passthrough connection).

A typical ICNN layer uses non-negative weights and convex, non-decreasing activations. If the weights $W_l^{(z)}$ are non-negative and the activation functions are convex and non-decreasing, then the composition preserves convexity of the output with respect to the input y.

For hyperelasticity, FFNN inputs are invariants of the Right Cauchy–Green tensor and pseudo-invariants for certain symmetries. The invariants I_C, II_C, III_C are convex functions of $\mathbf{F}$, $\operatorname{cof}\mathbf{F}$, $\det\mathbf{F}$ respectively, so a convex non-decreasing function of them guarantees polyconvexity.

3.5.6 Growth and Normalization Conditions

Linden et al. significantly advanced the ICNN framework by introducing the concept and nomenclature of Physics-Augmented Neural Networks (PANNs). While ICNNs with invariants as inputs and energy potential as output satisfy the requirements for thermodynamic consistency, objectivity, material symmetry, and material stability, they do not inherently satisfy the condition of a stress-free reference configuration or the energy normalization condition:

$$\mathbf{P}(\mathbf{I}) = \mathbf{0} \tag{3.9}$$

$$\psi(\mathbf{I}) = 0 \tag{3.10}$$

Where $\mathbf{I}$ represents the identity tensor (undeformed state where $\mathbf{F} = \mathbf{I}$, so $\mathbf{C} = \mathbf{I}$).

The implicit assumption is that ICNNs will learn these conditions from the training data, which leads to approximate satisfaction of these conditions (with non-zero residual stresses in the undeformed state). To solve this, growth and normalization terms are added to the original potential, decomposing the energy functional into an FFNN component and rigorous analytical correction terms:

$$\psi(\mathbf{C}) = \psi_{\mathrm{NN}}(\mathbf{C}) + \psi_{\mathrm{norm}}(\mathbf{C}) + \psi_{\mathrm{vol}}(J) \tag{3.11}$$

This new polyconvex, invariant-based stress normalization term ensures that the stress tensor vanishes at the reference configuration without violating the overall model's convexity requirements. Earlier work used penalty terms in the loss function (soft constraints) or simple correction terms. Linden et al. systematically derived polyconvex normalization terms rigorously for compressible materials. This leads to exact physics satisfaction, rather than approximate or adjusted learning.

3.6 Comparative Summary of Strategies and Outlook

3.6.1 Comparative Summary

As we have seen throughout this chapter, the field of machine learning-based constitutive modeling has moved past the naive assumption that a neural network, given enough data, will independently learn the laws of thermodynamics and mechanics. Instead, the most recent approaches present models constrained by the physics they represent.

To consolidate the methodologies from the three waves of development, Table 3.2 compares their core attributes side by side.

Table 3.2: High-level comparison of the primary strategies for physics enforcement in machine learning-based constitutive models.

Methodology	**Physics Enforcement**	**Interpretability**	**Data Requirements**	**Stability Guarantee**
Black-box FFNNs (1990s–2010s)	None	Low	High	None
PINNs (soft constraints)	Loss function	Low	High	Approximate
ICNNs / PANNs (hard constraints)	Architecture	Medium	Medium	**Exact**
Unsupervised discovery (EUCLID, ICKANs)	Loss + Architecture	High	Medium	**Exact**

3.6.2 Remarks and Interpretation

To close the chapter, we highlight two insights that follow from the literature reviewed above.

The first concerns the enforcement strategy. Moving from soft constraints

(loss-level penalties) to hard constraints (ICNNs, PANNs) addresses local stability violations in FE simulations. However, as the development of CSSV-NNs shows, overly restrictive architectural constraints can exclude valid physical behaviors (Geuken et al. 2025). The invariant-based ICNN formulation, for instance, cannot represent certain polyconvex energy functions that are linear in the principal stretches. The research challenge that remains open is defining architectural constraints whose admissible set matches the true boundary of physical admissibility, neither too narrow nor too broad.

The second concerns data. Classical supervised approaches need homogeneous stress states for training, which are difficult to obtain experimentally. Unsupervised methods such as EUCLID bypass this requirement entirely by embedding the neural network within a virtual work framework, backpropagating errors from globally measurable quantities directly into the local constitutive model.

Building on these developments, the field is advancing along two complementary directions. On one side, developing highly efficient and provably stable neural constitutive subroutines (PANNs) for direct integration into large-scale finite element simulations, where speed and numerical robustness are required. On the other, developing automated model discovery frameworks (EUCLID, symbolic regression) that treat the neural network as a temporary scaffold (once the network has captured the material behavior, we extract a closed-form analytical expression and the network becomes obsolete). These two directions are not mutually exclusive, as shown in the NN-EUCLID proposal.

3.6.3 Transition to Implementation

In this chapter, we have defined the theoretical requirements for a Physics-Augmented Neural Network (PANN): thermodynamic consistency, objectivity, material symmetry, polyconvexity, and normalization. In Chapter 4, we move from theory to implementation, translating the mathematical constraints from Section 3.5 into Python code. The result will be a constitutive model that is ready for integration into an FE solver.

Chapter 4

Methodology and Implementation

4.1 Problem Statement

4.1.1 Theoretical Setting

In this chapter, we describe the methodology for learning a constitutive law for compressible Neo-Hookean hyperelastic materials under the plane strain assumption using a Physics-Augmented Neural Network (PANN).

In general terms, we seek a model f such that:

$$\hat{\mathbf{S}} = f(\mathbf{E}) \tag{4.1}$$

where $\mathbf{E}$ is the Green–Lagrange strain tensor and $\mathbf{S}$ is the second Piola–Kirchhoff (PK2) stress tensor. Both quantities are expressed in Voigt notation under the plane strain restriction ($E_{13} = E_{23} = E_{33} = 0$), so that the input-output space is a map $\mathbb{R}^3 \to \mathbb{R}^3$:

$$\mathbf{E}_{\text{Voigt}} = [E_{11},\, E_{22},\, \gamma_{12}] \tag{4.2}$$

$$\mathbf{S}_{\text{Voigt}} = [S_{11},\, S_{22},\, S_{12}] \tag{4.3}$$

where we must note the use of the engineering shear strain $\gamma_{12} = 2E_{12}$.

The ground truth used to learn the model is generated by evaluating the compressible Neo-Hookean model implemented in Kratos Multiphysics (Dadvand, Rossi, and Oñate 2010), parametrized by Young's modulus $E = 10^7$ Pa and Poisson's ratio $\nu = 0.4$.

We compare the PANN with a black-box feedforward baseline that learns a model $f_{\boldsymbol{\theta}}$ such that:

$$\hat{\mathbf{S}} = f_{\boldsymbol{\theta}}(\mathbf{E}) \tag{4.4}$$

This baseline model will learn $\mathbf{S}$ directly, while the PANN will learn an intermediate strain energy density $\psi(\mathbf{C})$ from which we can derive stress via the thermodynamically consistent relation:

$$\mathbf{S}(\mathbf{C}(\mathbf{X}), \mathbf{X}) = 2\,\frac{\partial \psi(\mathbf{C}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{C}} = \frac{\partial \psi(\mathbf{C}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{E}} \tag{4.5}$$

where $\mathbf{C}$ is the right Cauchy–Green deformation tensor and $\mathbf{E}$ is the Green–Lagrange deformation tensor.

4.1.2 Objectives

With this implementation, we aim to directly address **RQ2:** *How does enforcing physical constraints by construction (in the neural network architecture) compare in terms of data efficiency and extrapolation capabilities with naive "black-box" learning or physics-informed models?*

As we will see in the following sections, the experiment has been designed to obtain exact error quantification using a well-known data distribution (generated from a Neo-Hookean simulation), and we intentionally avoid added complexity, such as using the network inside a FE solver loop.

The focus is on evaluating the PANN's behavior against the benchmark under data scarcity and heavy extrapolation requirements, to obtain clear insights into the benefits of physics-augmented models.

4.1.3 Software Organization and Dependencies

The implementation is organized in two separate repositories: `neohookean-data-generator` and `neohookean-ml-pipeline` for synthetic dataset generation and model development, training, and evaluation, respectively. Both are publicly available on GitHub[1] under the MIT License.

The data generation pipeline uses the KratosMultiphysics constitutive law API, while the model pipeline is built on TensorFlow, NumPy, and scikit-learn (*de facto* standard Python libraries for machine learning). Both codebases follow a modular structure separating data handling, model definitions, and evaluation utilities, so that individual components (e.g., the constitutive law backend or the network architecture) can be modified without affecting the rest of the workflow facilitating the work for future researchers. The full repository structure is provided in Appendix B, while dependencies are listed in Appendix C.

4.2 Synthetic Data Generation

4.2.1 Kratos API Implementation

As we just mentioned, the data generation pipeline has been implemented in the `neohookean-data-generator` repository. Here, the module `physics.py` sets up all the necessary Kratos environment so that we can isolate "physics-related" definitions and dependencies. For any student or researcher following this work, these definitions could eventually change without needing to change the rest of the pipeline if the input-output interfaces are consistent with the current workflow constraints.

To isolate the constitutive response from any structural, geometric, or boundary effects, we have made two relevant architectural decisions:

- First, **no finite element mesh, assembly, or equilibrium solve is involved in data generation**. Our (very minimalistic) approach is to

[1] https://github.com/jaguilamartinez

use a single `Triangle2D3` geometry over three nodes at $\{(0,0), (1,0), (0,1)\}$. With this approach, we satisfy the integration point interface that the Kratos constitutive law API demands without adding complexity to the overall exercise.

- Second, **stress vectors are generated by supplying a Green–Lagrange strain vector as input**, using the `USE_ELEMENT_PROVIDED_STRAIN = True` parameter to override the kinematic computation. To accommodate the Kratos internal state management, the three-step constitutive law evaluation protocol that returns PK2 in the Kratos API (`Initialize`, `Calculate`, `Finalize`) is executed per scenario evaluation, even though Neo-Hookean materials are path-independent. We could supply the different data points (strain vectors) independently, getting the same results.

With these two decisions, the finite element driver is transformed into a pure material-point evaluator, which will allow us to concentrate on modeling the constitutive relationship.

4.2.2 Strain Space Parametrization

The Voigt strain space $\mathbb{R}^3$ is parametrized using spherical coordinates (θ, ϕ, r) via the following map:

$$E_{11} = r \cdot \cos\theta \tag{4.6}$$

$$E_{22} = r \cdot \sin\theta \cdot \cos\phi \tag{4.7}$$

$$\gamma_{12} = 2 \cdot r \cdot \sin\theta \cdot \sin\phi \tag{4.8}$$

We include the factor of 2 in the shear component to preserve consistency with the Voigt convention, as we are using the engineering shear strain γ_{12} and not the tensorial component E_{12}.

With this transformation, the strain space is decomposed into two components:

- A directional component, $(\theta, \phi) \in \mathbf{S}^2$ (the loading mode)

- A magnitude component, $r \in \mathbb{R}_{\geq 0}$ (the loading magnitude)

In this new representation of the strain space, radial sweeps at fixed angles correspond to proportional loading paths (a single strain direction traversed at increasing intensity), while angular sweeps at fixed radius sample the full directional diversity at constant strain norm.

The implementation in `sphere_generator.py` supports two specification modes in the YAML configuration. Each of θ, ϕ, and r can be:

- A **scalar** (e.g., `theta: 0`): Fixed value, producing one point along that dimension.
- An **array** `[start, stop, n_points]` (e.g., `radius: [0, 0.25, 25]`): Generates `n_points` values via `np.linspace`, producing a sweep along that dimension.

This parameterization provides a systematic way to sample strain space.

All experiment parameters are specified in `config/experiment.yaml`, which defines:

- **Material**: Young's modulus (10^7 Pa) and Poisson's ratio (0.4).
- **Loading history**: 25 time steps (`n_steps`) per case, with linear ramping from zero to the target strain ($\alpha = \text{step}/(n_{\text{steps}} - 1)$).
- **12 named scenario blocks**: Each defines a name, spherical coordinate ranges, and optional flags. The 12 scenario types and their configurations are:
  1. **sphere** (362 cases): Full spherical sampling, $\theta \in [0°, 360°]$ (20 pts), $\phi \in [0°, 360°]$ (20 pts), $r = 0.25$, with pole-skipping to avoid duplicate cases at $\theta = 0°$ and $\theta = 360°$.
  2. **uniaxial_x** (25 cases): $\theta = 0°$, $\phi = 0°$, $r \in [0, 0.25]$.
  3. **uniaxial_x_compression** (25 cases): $\theta = 180°$, $\phi = 0°$, $r \in [0, 0.25]$.

4. **uniaxial_y** (25 cases): $\theta = 90°$, $\phi = 0°$, $r \in [0, 0.25]$.
5. **uniaxial_y_compression** (25 cases): $\theta = 270°$, $\phi = 0°$, $r \in [0, 0.25]$.
6. **equibiaxial** (25 cases): $\theta = 90°$, $\phi = 90°$, $r \in [0, 0.25]$.
7. **equibiaxial_compression** (25 cases): $\theta = 270°$, $\phi = 90°$, $r \in [0, 0.25]$.
8. **simple_shear** (25 cases): $\theta = 90°$, $\phi = 45°$, $r \in [0, 0.5]$.
9. **pure_shear** (25 cases): $\theta = 0°$, $\phi = 90°$, $r \in [0, 0.25]$.
10. **biaxial_2to1** (25 cases): $\theta = 63.43°$, $\phi = 90°$, $r \in [0, 0.25]$.
11. **phi_45_plane** (25 cases): $\theta \in [0°, 180°]$ (25 pts), $\phi = 45°$, $r = 0.25$.
12. **radial_45_45** (25 cases): $\theta = 45°$, $\phi = 45°$, $r \in [0, 0.5]$.

All combinations of the expanded parameters are generated as a Cartesian product, with a total of 637 cases for the 12 scenarios (362 for the sphere and 25 additional cases for each one of the 11 named scenarios).

Figure 4.1 provides a graphical representation of the 637 cases generated in the Voigt strain space.

4.2.3 Admissibility Filtering

Not every strain vector we generate will correspond to a deformation that could physically exist. The right Cauchy–Green tensor $\mathbf{C} = \mathbf{I} + 2\mathbf{E}$ must be symmetric positive definite (SPD) for the deformation to be physically realizable –if it is not, the implied deformation would collapse volume or produce negative stretches, neither of which can happen in a real material. Since we are working under plane strain, the out-of-plane component $C_{33} = 1 > 0$ (using $E_{33} = 0$) takes care of itself, so the SPD condition reduces to Sylvester's criterion on the 2×2 block:

$$C_{11} > 0 \tag{4.9}$$

$$C_{22} > 0 \tag{4.10}$$

$$C_{11}\, C_{22} - C_{12}^2 > 0 \tag{4.11}$$

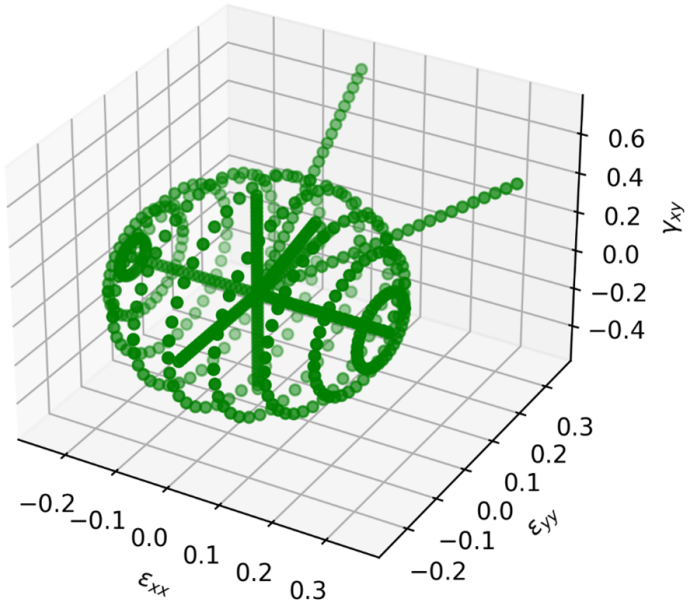

Figure 4.1: Spherical sampling of the admissible strain space in Voigt notation ($\mathbf{E}_{\text{Voigt}} = [E_{11}, E_{22}, \gamma_{12}]$). Each point on the sphere corresponds to a strain state generated via Kratos Multiphysics at a fixed angular position and variable radius, representing increasing deformation magnitude. The radial lines correspond to loading paths at fixed angular coordinates, sweeping from zero to the maximum radius.

Where $C_{11} = 1 + 2E_{11}$, $C_{22} = 1 + 2E_{22}$, and $C_{12} = 2E_{12} = \gamma_{12}$ (the Voigt convention absorbs the factor of 2).

The function `is_admissible_GL_plane_strain()` in the pipeline enforces these with tolerance $\varepsilon = 10^{-8}$.

4.2.4 Dataset Structure

The final dataset is created via linear ramps (sweeps) over the target strain. As described earlier, sweeps can be done over the directional and/or magnitude components. Each sweep consists of proportional increments of $n = 25$ steps, so that

$$\mathbf{E}_k = \frac{k}{n-1}\,\mathbf{E}_{\text{target}} \tag{4.12}$$

for $k = 0, \ldots, 24$.

As mentioned in Section 4.1, in a Neo-Hookean model (hyperelastic, path-independent), every intermediate point is a valid strain–stress pair, so this scheme of incremental generation amplifies the dataset by a factor of 25.

The consolidated dataset has, for this reason, 15,925 samples (637×25). Every sample is stored including the `strains`, `stresses`, `case_ids`, and `scenario_labels` in a `.npz` file called `consolidated_all.npz`.

4.2.5 Complete Data Generation Pipeline

The generation script executes the following steps:

1. **Load configuration**: We start by parsing the YAML file to extract material parameters, step count, and scenario definitions.
2. **Generate cases**: We then call `generate_cases_from_config()` which expands all parameter sweeps, computes strain vectors via the spherical mapping, and filters out inadmissible strains.
3. **Initialize Kratos**: We follow by calling `setup_kratos_model()` to create the finite element model, constitutive law, properties, and geometry once.

4. **Loop over all cases**: For each case, we linearly ramp the strain from zero to the target over `n_steps` steps. At each step, we call `calculate_stress_from_strain()` to obtain the PK2 stress. This produces a `(n_steps, 3)` strain history and a `(n_steps, 3)` stress history per case.

5. **Save per-case data**: Each case is saved as an individual `.npz` file under `output/<scenario_name>/raw_data/`, containing the full strain and stress histories plus metadata (scenario name, θ, ϕ, r, case number).

6. **Generate plots**: Finally, we generate per-case stress–strain curves and 3D strain trajectory plots to save them alongside the raw data.

Figure 4.2 presents an example plot for a singular stress–strain curve corresponding to one of the points in the surface of the sphere.

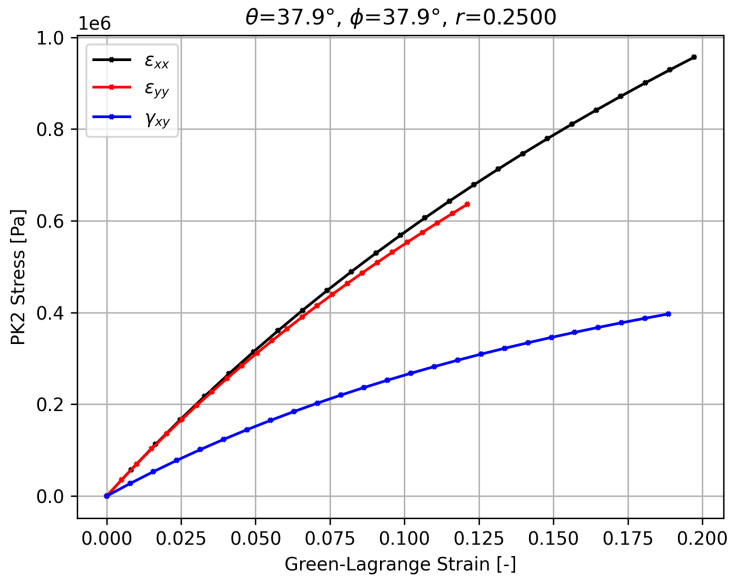

Figure 4.2: Example stress–strain curve for a single loading case on the surface of the sphere.

After generation, we aggregate all individual case results into a single dataset. Each sample is tagged with a unique case identifier and its scenario label (scenario name), and all strain–stress pairs across all time steps and cases are stacked into flat numpy arrays. The result is a single consolidated file containing all 15,925

samples, ready for use as the training and evaluation dataset in the next step of the process.

4.3 Data Management in the ML Pipeline

4.3.1 Case-Based Splitting

The standard recipe for evaluating generalization in machine learning is to split the data into training, validation, and test sets. However, our data has a structure that makes a naive random split dangerous: samples along the same proportional loading path are identical or nearly identical neighbours in strain space, so if one falls in the training set and its sibling in the test set, the model can score well by interpolating along a trajectory it has already memorized – a textbook case of data leakage. This would produce misleadingly optimistic evaluation metrics, especially for highly expressive models such as the base neural network.

To avoid this, the function `prepare_data_by_groups()` splits the consolidated dataset at the *case* level, so the 637 unique case IDs are partitioned into disjoint training/validation/test subsets (60%/20%/20%), and each case is assigned to a single partition.

For the generalization experiments, splitting is performed at the *scenario* level, so entire strain type categories (e.g., equibiaxial, uniaxial) are not present in the training set, ensuring a strict out-of-distribution evaluation.

4.3.2 Input Normalization

In machine learning, inputs are usually normalized to facilitate faster convergence during training. This normalization can be done by using a `StandardScaler` (adjusting the data distribution so that in every feature $\mu = 0$ and $\sigma = 1$) or using a `MinMaxScaler` (adjusting the range of the data to the range $[0, 1]$).

A hard-won lesson of this implementation is that the PANN operates on unscaled physical quantities. The PANN's forward pass computes $\mathbf{C} = \mathbf{I} + 2\mathbf{E}$,

whose physical meaning depends on the identity tensor representing the undeformed metric. Normalizing $\mathbf{E}$ shifts this anchor point to a non-zero value, which means the invariants no longer represent what they should, and the reference-state corrections (designed to ensure zero stress at $\mathbf{C} = \mathbf{I}$) break. In our original implementation, the model trained and the loss decreased, but the predicted stresses were offset by a constant that made no physical sense. The fix was to keep the inputs at their physical scale and instead introduce a learnable energy scale parameter that enforces the physical stress magnitudes ($\sim 10^5$–10^6 Pa).

For the baseline model, we have decided to operate on normalized inputs and outputs, facilitating the black-box model's learning process. This decision aligns with keeping the baseline model "physics-agnostic" and follows standard machine learning practice.

4.4 Baseline Model

The baseline model (`base_model.py`) is a conventional black-box neural network for direct strain-to-stress regression. It learns the mapping $\hat{\mathbf{S}} = f_{\boldsymbol{\theta}}(\mathbf{E}_{\text{Voigt}})$ directly, with no embedded knowledge of physics. For the implementation, we have used a Keras `Sequential` model with three hidden layers of widths 128, 64, and 32. In each hidden layer, we apply the following sequence:

1. Dense layer with ReLU[2] activation
2. `BatchNormalization` for training stability
3. `Dropout(0.2)` for regularization

The output layer is a linear `Dense` layer that directly produces the three Voigt stress components.

The model is compiled with the Adam optimizer (default learning rate 10^{-3}) and Mean Squared Error (MSE) loss. Training employs two Keras callbacks:

[2]ReLU is standard in generic ML but would be unsuitable for an energy-based model, which is precisely the point of the comparison.

- `EarlyStopping` monitoring validation loss with patience of 20 epochs and best-weights restoration.
- `ReduceLROnPlateau` halving the learning rate after 10 epochs of no improvement, down to a minimum of 10^{-7}.

Note that there is no particular rationale behind selecting this architecture, other than it being a "standard" architecture commonly used in machine learning. This model knows nothing of frame indifference, material symmetry, or energy potentials, serving as the reference baseline against which the PANN's inductive bias is quantified. Both models learn the same function from the same data, so any performance gap can be attributed to the physics-augmented architecture of the PANN that we describe in the next section.

4.5 Physics-Augmented Neural Network (PANN)

4.5.1 Definition of the Output (Ensuring Thermodynamic Consistency)

As we have described in Section 4.1, in hyperelastic materials we assume a stored energy density $\psi(\mathbf{C}(\mathbf{X}), \mathbf{X})$ such that

$$\mathbf{S}(\mathbf{C}(\mathbf{X}), \mathbf{X}) = 2\,\frac{\partial \psi(\mathbf{C}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{C}} = \frac{\partial \psi(\mathbf{C}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{E}} \tag{4.13}$$

Designing a network to predict ψ rather than $\mathbf{S}$ directly enforces some properties by construction. First, stress depends only on the current deformation state, and not on the loading history. Second, the resulting elasticity tensor $\mathbb{C} = 2\,\partial \mathbf{S}/\partial \mathbf{C} = 4\,\partial^2 \psi/\partial \mathbf{C}^2$ is automatically symmetric ($\mathbb{C}_{ijkl} = \mathbb{C}_{klij}$) by equality of mixed partial derivatives (a direct mapping offers no guarantee of tangent symmetry). Finally, the existence of a scalar potential ψ ensures that the material stores and releases energy without dissipation, which is the necessary and sufficient condition for thermodynamic consistency.

In practice, this means that the network will learn a single scalar output ψ, from which the three Voigt stress components will be obtained via automatic differentiation.

4.5.2 Definition of the Input (Ensuring Frame Indifference)

As described in Chapter 3, using invariants allows us to learn within a representation that guarantees objectivity. This can be expressed in our implementation as defining a set of inputs such that:

$$\psi = \hat{\psi}(\mathrm{I}_C, \mathrm{II}_C, \mathrm{III}_C) \tag{4.14}$$

With:

- $\mathrm{I}_C = \mathrm{tr}(\mathbf{C}) = \lambda_1^2 + \lambda_2^2 + \lambda_3^2$
- $\mathrm{II}_C = \mathrm{tr}(\mathrm{cof}\,\mathbf{C}) = \lambda_1^2\lambda_2^2 + \lambda_2^2\lambda_3^2 + \lambda_3^2\lambda_1^2$
- $\mathrm{III}_C = \det\mathbf{C} = J^2 = (\lambda_1\lambda_2\lambda_3)^2$

This means that we use the three scalar invariants of $\mathbf{C}$ rather than the three (non-trivial) components of the right Cauchy–Green tensor under plane strain, encoding material symmetry and frame indifference by construction and ensuring that, for any rotation $\mathbf{Q}$, $\psi(\mathbf{Q}\mathbf{C}\mathbf{Q}^\mathsf{T}) = \psi(\mathbf{C})$, we get to the same stress state without requiring data augmentation or penalty terms to embed frame indifference.

Following Linden et al. (2023), we include a fourth term $-2J$ alongside the three principal invariants ($[\mathrm{I}_C, \mathrm{II}_C, \mathrm{III}_C]$). This fourth feature is redundant in terms of information ($J = \sqrt{\mathrm{III}_C}$), but it serves two purposes: (a) it avoids requiring the ICNN to learn the highly nonlinear square root map $\sqrt{\mathrm{III}_C}$ through non-negative weights and softplus activations, and (b) its specific coefficient (-2) makes the stress correction coefficient c computable as a simple weighted sum of the invariant-space gradient, without requiring separate chain rule terms for J.

4.5.3 Forward Pass Architecture

The PANN transforms a plane strain Green–Lagrange strain vector into a stress prediction through five sequential stages. Figure 4.3 illustrates this data flow.

The input is the Voigt strain vector, $\mathbf{E}_{\text{Voigt}} = [E_{11}, E_{22}, \gamma_{12}]$. From this, we reconstruct the right Cauchy–Green tensor via $\mathbf{C} = \mathbf{I} + 2\mathbf{E}$ converting γ_{12} to $E_{12} = \gamma_{12}/2$ and setting all out-of-plane components to zero (plane strain).

From $\mathbf{C}$, we compute the three principal invariants (I_C, II_C, III_C) as defined in Section 2.2.3. These scalars form the input to the neural network, ensuring frame indifference by construction (see Section 3.5.3).

The invariants are then passed through an Input-Convex Neural Network (ICNN), which will be described in Section 4.5.4. The ICNN outputs a raw scalar energy ψ_{ICNN}.

This raw prediction is then corrected to satisfy the normalization and growth conditions described in Section 3.5.6. Specifically, we subtract the reference-state energy to enforce $\psi(\mathbf{I}) = 0$ and $\mathbf{P}(\mathbf{I}) = \mathbf{0}$, and we add a volumetric growth term to ensure stability under large deformations. The result of this step is the total strain energy, ψ_{PANN}. The correction terms will be described in Section 4.5.5.

Finally, we can obtain the second Piola–Kirchhoff stress, $\mathbf{S}$, by automatic differentiation of ψ_{PANN} with respect to $\mathbf{C}$ using $\mathbf{S} = 2\,\partial\psi/\partial\mathbf{C}$ as described in Section 2.2.5.

4.5.4 Implementation of the ICNN

As we can see in Figure 4.3, the PANN is implemented in two modules: an outer model that handles the kinematics (computing $\mathbf{C}$ from $\mathbf{E}_{\text{Voigt}}$), the corrections, and the stress differentiation, and an inner model that implements the ICNN predicting the strain energy from the invariants. This section focuses on the ICNN, while the following section will describe the different corrections made to the raw strain energy, ψ_{ICNN}, to get the final strain energy, ψ_{PANN}.

As discussed in Section 3.5.5, polyconvexity of the strain energy requires the

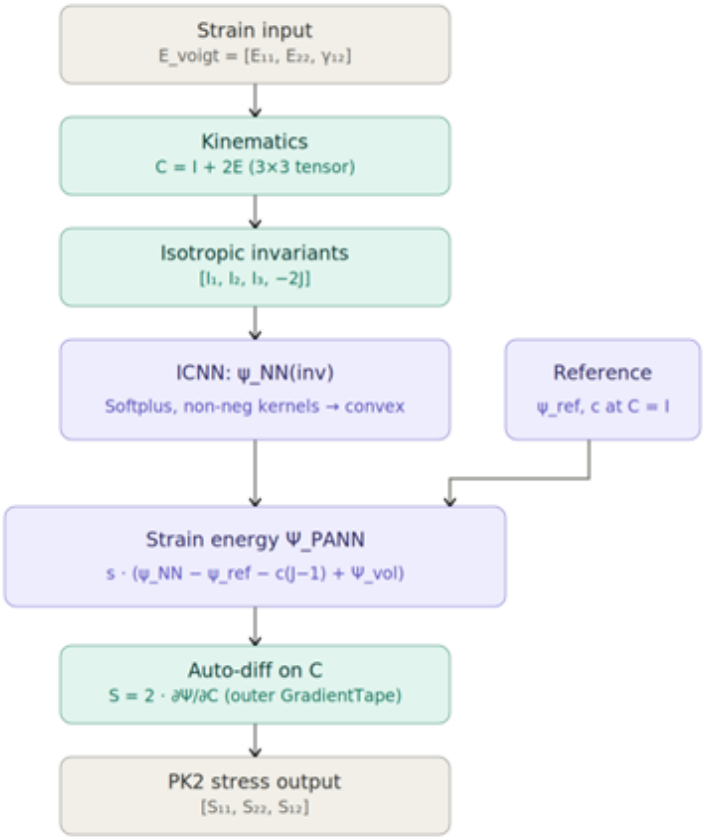

Figure 4.3: Forward pass of the PANN: Green–Lagrange strain, $\mathbf{E} \rightarrow$ right Cauchy–Green tensor, $\mathbf{C} \rightarrow$ isotropic invariants, $[\mathrm{I}_C, \mathrm{II}_C, \mathrm{III}_C, -2J] \rightarrow$ ICNN energy, ψ_{ICNN}, combined with a reference-state correction, ψ_{ref} (c at reference state) and volumetric penalty, ψ_{vol}, into the total strain energy, ψ_{PANN}, differentiated with respect to $\mathbf{C}$ via automatic differentiation to produce the PK2 stress, $\mathbf{S} = 2\,\partial\psi/\partial\mathbf{C}$.

network output to be convex with respect to its inputs. We enforce this through two architectural constraints. First, all inter-layer weight matrices are restricted to non-negative values, projecting to zero any negative weight after the gradient update. Second, every hidden neuron uses the softplus activation function ($\sigma(x) = \ln(1+e^x)$), which is smooth, convex and monotonically non-decreasing. Applying a convex, non-decreasing function to a convex function preserves convexity (Amos, Xu, and Kolter 2017). This means that convexity "propagates" through the network by induction (if the input is a convex function of the invariants, we can guarantee that output will also be convex).

The ICNN consists of two hidden layers of width 16 (considerably smaller than the baseline's 128-64-32 architecture) with softplus activations, followed by a single-output layer that also uses softplus, producing the scalar energy, ψ_{ICNN}.

4.5.5 Correction Terms

As described in Section 3.5.6, a physically admissible strain energy must satisfy the energy datum ($\psi(\mathbf{I}) = 0$) and zero stress ($\mathbf{S}(\mathbf{I}) = \mathbf{0}$) at the undeformed reference configuration. The raw ICNN output, ψ_{ICNN}, satisfies neither condition in general, so we apply the correction terms introduced by Linden et al. (2023). These corrections are applied at every forward pass as hard-coded corrections, not as penalty terms in the loss, enforcing both conditions *exactly* regardless of the network's weights.

1. **Energy normalization correction:** We subtract the ICNN energy evaluated at the reference configuration:

$$\psi_{\text{norm}} = \psi_{\text{ICNN}}(\mathbf{C}) - \psi_{\text{ICNN}}(\mathbf{I}) \tag{4.15}$$

   This guarantees $\psi(\mathbf{I}) = 0$ by construction, shifting the energy surface so that the undeformed state sits at zero without changing the gradients of the energy.

2. **Stress-free reference correction:** A first-order correction ensures that the gradient of ψ vanishes at $\mathbf{C} = \mathbf{I}$, so that the second Piola–Kirchhoff stress is

zero in the undeformed configuration. This term is derived analytically from the ICNN's gradient at the reference state, following the procedure described in Linden et al. (2023).

3. **Volumetric growth penalty:** We add an additional term $\psi_{\text{vol}}(J)$, defined as:
$$\psi_{\text{vol}}(J) = \left(J + \frac{1}{J} - 2\right)^2 \tag{4.16}$$
This term regularizes the volumetric response independently of the ICNN. It is zero at $J = 1$, diverges as $J \to 0$ and $J \to \infty$, and has vanishing first derivative at $J = 1$ so it does not interfere with the stress-free condition.

4. **Energy scaling:** Finally, the ICNN's raw output is bounded by the non-negative weight constraint and softplus activations, so its magnitude is much smaller than the physical stress scale. To address this, we use a trainable scalar λ that multiplies the corrected energy, allowing the network to match physical stress magnitudes without input or output normalization.

The assembled energy is:

$$\psi(\mathbf{C}) = \lambda \cdot [\psi_{\text{ICNN}}(\mathbf{C}) - \psi_{\text{ICNN}}(\mathbf{I}) - c\,(J-1) + \psi_{\text{vol}}(J)] \tag{4.17}$$

4.5.6 Stress Differentiation

Finally, we obtain the second Piola–Kirchhoff stress via $\mathbf{S} = 2\,\partial\psi/\partial\mathbf{C}$ using TensorFlow `GradientTape`, which records all operations performed on $\mathbf{C}$ during the forward pass (invariant computation, ICNN evaluation, corrections) and then applies the chain rule in reverse to compute $\partial\psi/\partial\mathbf{C}$ exactly, following the automatic differentiation mechanism described in Section 2.3.5. This provides three advantages over numerical differentiation: the derivative is exact to machine precision, requires no finite-difference approximation, and its computational cost is proportional to a single forward evaluation of ψ.

Note that we differentiate through the right Cauchy–Green tensor $\mathbf{C}$ and not through the Green–Lagrange tensor $\mathbf{E}$. Using $\mathbf{C}$ and not $\mathbf{E}$ and then extracting

Voigt components is the standard formulation in continuum mechanics and is particularly effective to avoid notation-dependent corrections such as the factor of 2 between the engineering shear strain, γ_{12}, and the tensorial component, E_{12}.

4.6 Training Process

Both models are trained with identical training protocol to ensure a fair comparison. The loss function is the MSE on the three Voigt components of the second Piola–Kirchhoff stress, optimized with the same Adam configuration as the baseline. Training uses early stopping with a patience of 20 epochs (restoring the weights with the lowest validation loss) and learning rate reduction by a factor of 0.5 after 10 epochs without improvement.

Although the PANN's architecture routes through the scalar energy ψ, the supervised signal is the stress $\mathbf{S}$, not the energy itself. Because the automatic differentiation of ψ is embedded in the forward pass, the MSE loss on stress implicitly shapes the energy landscape. This is a form of Sobolev training (Section 2.3.5): the loss penalizes the derivative of the learned function rather than the function itself, which provides stronger supervision because it constrains the shape of ψ, not just its values.

Chapter 5

Numerical Results and Discussion

5.1 Introduction

In this chapter, we compare the two architectures introduced in Chapter 4 for predicting the second Piola–Kirchhoff (PK2) stress tensor in an isotropic, Neo-Hookean material under plane strain conditions. As a refresher, the two architectures in play are the following:

1. **Baseline:** A conventional feedforward network ($128 \to 64 \to 32$, with BatchNorm and Dropout) that learns a direct mapping from Green–Lagrange strain to PK2 stress in scaled space.

2. **PANN:** A Physics-Augmented Neural Network built on an Input Convex Neural Network (ICNN) that learns a strain energy density function ψ through computing the invariants of the right Cauchy–Green tensor $\mathbf{C}$ and derives stress through automatic differentiation:

$$\mathbf{S}(\mathbf{C}(\mathbf{X}), \mathbf{X}) = 2\,\frac{\partial \psi(\mathbf{C}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{C}} \tag{5.1}$$

Through this analysis, we plan to address **RQ2:** How does enforcing physical constraints by construction (in the neural network architecture) compare in terms of data efficiency and extrapolation with naive "black box" learning?

The experiment covers five scenario-based train/test splits (testing interpolation vs. extrapolation under varying deformation-mode coverage), five data-scarcity experiments (testing sample efficiency), and detailed per-component error analysis.

All experiments use the same dataset, described in the next section, the same optimizer (Adam), and the same training policies (early stopping and learning rate reduction calls, with up to 150 epochs per run).

5.2 Description of the Dataset

The dataset has been generated using Kratos Multiphysics (Dadvand, Rossi, and Oñate 2010) from a Neo-Hookean law, producing pairs of a given Green–Lagrange strain and the corresponding PK2 stress under plane strain conditions. For both the strain and the stress we are using the Voigt notation, with $\mathbf{E}_{\text{Voigt}} = [E_{11}, E_{22}, \gamma_{12}]$ (where $\gamma_{12} = 2E_{12}$) and $\mathbf{S}_{\text{Voigt}} = [S_{11}, S_{22}, S_{12}]$.

As described in Chapter 4, we have organized the dataset into 12 named loading scenarios with 637 different cases that create a set of 15,925 samples through a schema of incremental generation in 25 steps:

5.3 Default Scenario

5.3.1 Data Split

In this first experiment, we analyze generalization performance with models trained on a diverse dataset. The evaluation strategy is built on a two-stage split that separates interpolation from extrapolation, as defined by the scenarios we previously discussed.

In the first stage, we select seven scenario types for training: uniaxial tension and compression in both axes, biaxial 2:1, pure shear, and simple shear (a total of 4,375 samples across 175 unique cases). These training scenarios are split internally by case (not by sample) into training (60% – 105 cases and 2,625 samples), validation (20% – 35 cases and 875 samples), and test (20% – 35 cases and 875 samples). This

Table 5.1: Number of Cases and Samples in the dataset. There are 12 named scenarios corresponding to different strain modes. Each one defines a set of cases by sweeping the directional or magnitude component. For each case, we generated augmentations incrementally over 25 steps, resulting in 637 cases and 15,925 samples.

Scenario	**Cases**	**Samples**
uniaxial_x	25	625
uniaxial_y	25	625
uniaxial_x_compression	25	625
uniaxial_y_compression	25	625
biaxial_2to1	25	625
pure_shear	25	625
simple_shear	25	625
equibiaxial	25	625
equibiaxial_compression	25	625
phi_45_plane	25	625
radial_45_45	25	625
sphere	362	9,050
total	**637**	**15,925**

test set, which we will refer to as **seen-test**, will allow us to understand how the model performs on deformation modes it has been trained on, measuring its ability to **interpolate**.

In the second stage, we use all the samples in the remaining five scenarios as held-out cases: sphere, equibiaxial tension and compression, the $\phi = 45°$ meridian, and the 45°–45° radial path (a total of 11,550 samples across 462 cases). This test set, which we will refer to as **unseen-test**, contains entirely new deformation modes that the model has never seen, measuring its ability to **extrapolate**.

5.3.2 Interpolation Performance (Seen Scenarios)

As described in Section 5.3.1, the seen-test set contains 875 samples from the seven loading modes used during training, drawn from held-out cases. Table 5.2 summarizes the aggregate metrics.

Table 5.2: Aggregate interpolation metrics on the seen-scenario test set (875 samples from 7 training loading types). Both models are evaluated on held-out cases not used during training.

Metric	Baseline	PANN	Improvement
R^2	0.9944	0.9979	+0.0035
RMSE (Pa)	23,725	14,425	39.2%
MAE (Pa)	16,436	7,638	53.5%

Both models achieve an R^2 value above 0.99, confirming that they can learn the constitutive map when exposed to sufficient data from the target deformation modes. The PANN demonstrates better performance, reducing the RMSE by 39.2% and the MAE by 53.5% relative to the baseline.

Table 5.3 summarizes the per-component breakdown.

The shear component S_{12} is where the architectural difference becomes most visible, with a 66.0% reduction in MAE. The follows directly from how the models represent the output (stress): the PANN learns a single scalar ψ and derives the three stress components by differentiation, while the baseline learns three independent

Table 5.3: Per-component interpolation metrics on the seen-scenario test set (875 samples from 7 training loading types). Both models are evaluated on held-out cases not used during training.

Component	Metric	Baseline	PANN	Improvement
S_{11}	R^2	0.9935	0.9975	+0.0040
S_{11}	RMSE	26,349 Pa	16,302 Pa	38.1%
S_{22}	R^2	0.9924	0.9969	+0.0044
S_{22}	RMSE	27,086 Pa	17,425 Pa	35.7%
S_{12}	R^2	0.9973	0.9994	+0.0022
S_{12}	MAE	10,547 Pa	3,590 Pa	66.0%

outputs (nothing in the architecture prevents the network from predicting a shear stress that is inconsistent with its own normal stress predictions). If the training data happens to be dominated by uniaxial cases (as ours is), the network receives weak supervision on shear and learns it poorly. The PANN does not suffer from this because shear information is embedded in the same energy function used to train the normal stresses. The shear information, in a sense, comes for free.

5.3.3 Extrapolation Performance (Unseen Scenarios)

This is, without any doubt, the most important test for the PANN's physical constraints. The unseen-test set contains 11,550 samples from five loading types that were not included in the training set: sphere, equibiaxial tension and compression, the $\phi = 45°$ meridian, and the 45°–45° radial path.

Table 5.4 summarizes the aggregate metrics.

In this case, the baseline performance drops with respect to the seen-test results, with R^2 going down from the 0.9944 on seen deformation modes (see Table 5.2) to 0.7284, a degradation of over 26 percentage points. Both the RMSE and MAE for the baseline are on the order of hundreds of kPa (368,274 Pa and 192,435 Pa, respectively), which makes the model virtually unusable in any practical scenario.

Table 5.4: Aggregate extrapolation metrics on the unseen-scenario test set (11,550 samples from 5 unseen loading types).

Metric	Baseline	PANN	Improvement
R^2	0.7284	0.9919	+0.2635
RMSE (Pa)	368,274	66,826	81.9%
MAE (Pa)	192,435	30,888	83.9%

The PANN, by contrast, maintains its performance, with a value of R^2 above 0.99 and a degradation of only 0.6 percentage points relative to its seen-scenario performance (0.9919 in unseen-test vs 0.9979 in seen-test). Its RMSE and MAE values are larger than those obtained on seen data (66,826 Pa and 30,888 Pa, respectively), but remain within a reasonable range.

Table 5.5 summarizes the per-component breakdown for the R^2:

Table 5.5: Per-component extrapolation R^2 on the unseen-scenario test set (11,550 samples from 5 unseen loading types).

Component	Baseline	PANN	Improvement
S_{11}	0.7929	0.9905	+0.1976
S_{22}	0.7401	0.9881	+0.2480
S_{12}	0.6523	0.9972	+0.3449

The PANN maintains R^2 above 0.98 on all three components, while the baseline drops below 0.80 on both normal components and below 0.66 on shear. The gap is widest on S_{12} where the PANN outperforms the baseline by 0.34 points. This is the same component that showed the largest improvement in the seen data (Table 5.2), reinforcing the argument for the advantage of the PANN's coupled learning mechanism.

5.3.4 Generalization Gap (Seen vs Unseen Scenarios)

Table 5.6 quantifies the generalization gap, measured as the drop/ratio in unseen scenarios relative to the seen-scenario baseline.

Table 5.6: Generalization gap, measured as performance in the seen vs unseen test sets, for the different metrics.

Metric	Model	Seen	Unseen	Drop / Ratio
R^2	Baseline	0.9944	0.7284	−0.2660
R^2	PANN	0.9979	0.9919	−0.0060
RMSE (Pa)	Baseline	23,725	368,274	15.52×
RMSE (Pa)	PANN	14,425	66,826	4.63×
MAE (Pa)	Baseline	16,436	192,435	11.71×
MAE (Pa)	PANN	7,638	30,888	4.04×

As shown in Sections 5.3.2 and 5.3.3, the baseline's R^2 degrades by 26.6 percentage points from its seen-scenario value when used in new deformation modes, while the PANN degrades by 0.6 percentage points (a 97.7% reduction in the generalization gap).

The RMSE and MAE ratios are aligned with these results, with the baseline's errors inflating by factors of 15.52 and 11.71, respectively, while the PANN's errors inflate by factors of 4.63 and 4.04, respectively.

Table 5.7 breaks down the generalization gap by stress component.

The baseline's degradation is relevant in the three components, while the PANN's performance is stable with an R^2 over 0.99 across the different stress measures. The ordering of degradation across the different stress measures in the baseline reflects the information content of the training scenarios, with much more information about normal stresses than about shear, leaving the baseline with insufficient training signal for the shear component. In the PANN, the uniform degradation reflects the coupling effect we introduced in Section 5.3.2 (since all components derive from the same scalar energy, degradation in the prediction

Table 5.7: Per-component generalization gap, measured as performance in the seen vs unseen test sets, for R^2 and RMSE.

Component	Model	R^2 (seen)	R^2 (unseen)	R^2 drop	RMSE ratio
S_{11}	Baseline	0.9935	0.7929	0.2007	15.11×
S_{11}	PANN	0.9975	0.9905	0.0070	5.24×
S_{22}	Baseline	0.9924	0.7401	0.2523	12.15×
S_{22}	PANN	0.9969	0.9881	0.0088	4.04×
S_{12}	Baseline	0.9973	0.6523	0.3450	23.17×
S_{12}	PANN	0.9994	0.9972	0.0023	4.57×

affects them proportionally).

5.3.5 Qualitative Stress–Strain Response

Figures 5.1–5.4 plot the predicted second Piola–Kirchhoff stress against the Kratos reference along four loading paths from the sampled strain space. Markers denote the ground truth, solid lines the PANN, and dashed lines the baseline. Color identifies the stress component (S_{xx}, S_{yy}, S_{xy}).

Both models reproduce the dominant component of each mode, and the errors concentrate in the secondary components. There the baseline departs from the reference, while the PANN stays close to it.

The simple-shear path (Figure 5.1) is the clearest case. The reference S_{yy} rises to a maximum near $\gamma_{xy} \approx 0.25$ and then falls, but the baseline predicts a stress that increases throughout and misses the turning point. As mentioned before, the baseline maps strain to stress directly and fits each component independently, while the PANN obtains all components from the gradient of a single energy potential, which here recovers the descent. The off-axis paths (Figures 5.3 and 5.4) show the same pattern in milder form. In Figure 5.4 the PANN over-predicts the normal components, but it remains closer to the reference than the baseline in every panel.

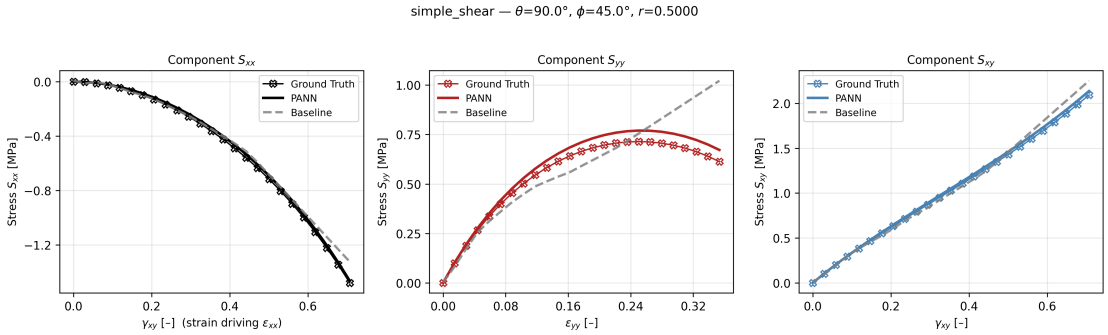

Figure 5.1: Second Piola–Kirchhoff stress along a simple-shear path ($\theta = 90°$, $\phi = 45°$, $r = 0.5$). The reference S_{yy} reaches a maximum near $\gamma_{xy} \approx 0.25$ and then decreases; the baseline keeps rising, while the PANN follows the peak.

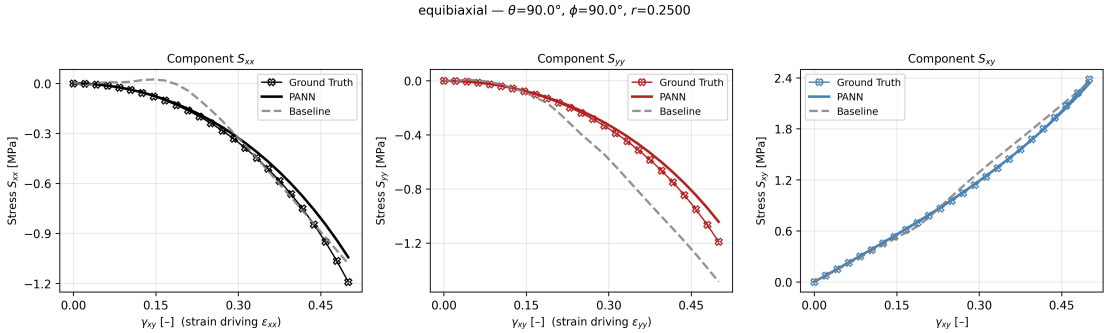

Figure 5.2: Second Piola–Kirchhoff stress along an equibiaxial path ($\theta = 90°$, $\phi = 90°$, $r = 0.25$). The PANN matches all three components; the baseline over-predicts S_{yy}.

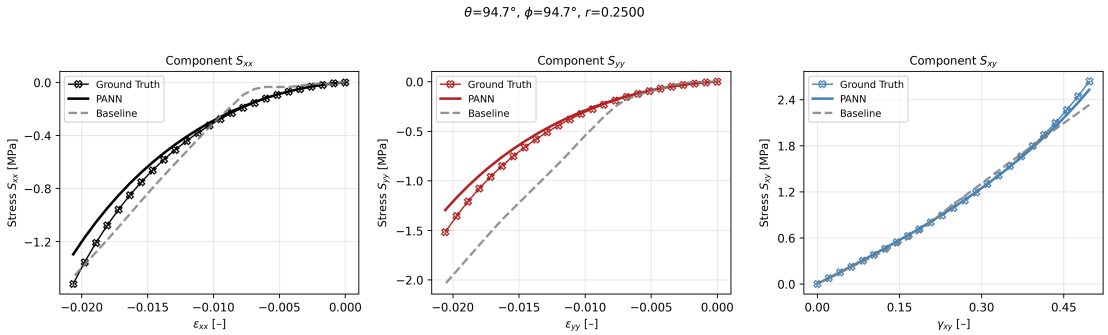

Figure 5.3: Second Piola–Kirchhoff stress along an off-axis path ($\theta = \phi = 94.7°$, $r = 0.25$). The baseline over-predicts the compressive S_{yy}; the PANN follows the reference in all components.

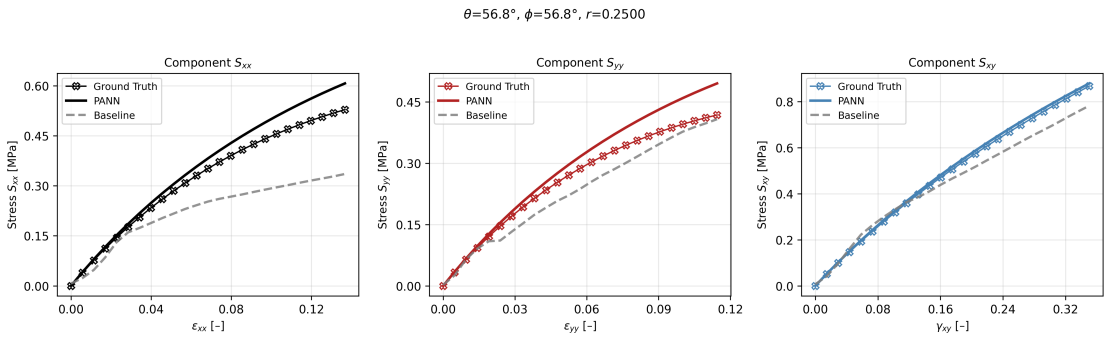

Figure 5.4: Second Piola–Kirchhoff stress along an off-axis path ($\theta = \phi = 56.8°$, $r = 0.25$). The baseline under-predicts every component; the PANN tracks the reference, with a small over-prediction of the normal components.

5.4 Sensitivity to Training Data Composition

5.4.1 Alternate Scenario Splits

The results in Section 5.3 were obtained with a fixed data split (seven scenarios). A natural question is whether the observed patterns are specific to this split or represent architectural properties of each architecture, and how the quality of the data representation in the training set affects the predictions. In this sense, we have designed a set of experiments with five scenario-based splits with increasingly favorable conditions:

1. **Uniaxial Only (4 train / 8 test):** We train the model on uniaxial tension in X, uniaxial tension in Y, uniaxial compression in X, and uniaxial compression in Y, and test on all other configurations. This is clearly the hardest extrapolation test, as the training set contains only uniaxial deformation paths, and the model must then generalize to biaxial, shear, and mixed loading.

2. **Uniaxial + Shear (6 train / 6 test):** We add pure shear and simple shear to the training set. This provides the model with, at least, some exposure to non-zero S_{12}.

3. **Original (7 train / 5 test):** This is the default split we used in the previous section. We add a 2:1 biaxial tension scenario to the previous one,

while keeping the rest for testing (spherical sampling, equibiaxial tension, equibiaxial compression, the $\phi = 45°$ meridian sweep, and the 45°–45° radial path).

4. **Diverse (6 train / 6 test):** This is a diverse training set, including uniaxial tension in X, uniaxial compression in Y, 2:1 biaxial tension, simple shear, equibiaxial tension, and $\phi = 45°$ to sample broadly across deformation modes.

5. **Leave Sphere Out (11 train / 1 test):** Train on all 11 deterministic scenarios. Test only on the sphere scenario, which densely covers the strain space but was never seen during training.

In each case, we evaluate interpolation (seen scenarios) and extrapolation (unseen scenarios) with the same methodology used in Section 5.3.

5.4.2 Generalization Gap for Alternate Scenario Splits

Table 5.8 summarizes R^2 on seen and unseen scenarios for each split configuration.

Table 5.8: R^2 on seen and unseen scenarios across five training splits of increasing diversity. The gap column reports the difference between seen and unseen R^2.

	Seen R^2		**Unseen R^2**		**Gap**	
Split	Baseline	PANN	Baseline	PANN	Baseline	PANN
Uniaxial only	0.664	0.850	0.571	0.745	0.094	0.105
Uniaxial + shear	0.993	0.993	0.738	0.978	0.255	0.015
Original (7 scen.)	0.994	0.998	0.728	0.992	0.266	0.006
Diverse (6 scen.)	0.988	0.993	0.844	0.978	0.144	0.015
Leave-sphere-out	0.992	0.985	0.943	0.971	0.049	0.013

As we can see in the table, the baseline's generalization gap is highly sensitive to the training diversity, with unseen R^2 values that range from 0.571 to 0.943 depending on how many scenarios are included in the training set. In the uniaxial only scenario, the R^2 for seen scenarios (0.664) shows that the baseline model has

not been able to properly fit the training data, keeping a generalization gap of 9.4 percentage points – a case of underfitting. From this point, adding more scenarios reduces the generalization gap, with the two extremes being the uniaxial + shear scenario (25.5 percentage points of generalization gap) and the leave-sphere-out scenario (4.9 percentage points of generalization gap, even with 11 of 12 scenarios included in the training data).

The PANN's gap is nearly constant at 1.5 percentage points, with the only exception being the uniaxial-only scenario, where the composition of the training set (only 4 deformation modes, none containing shear) is insufficient to train the model, which does not fully converge (training runs all 150 epochs with the loss still decreasing). Even in this extreme case, however, it outperforms the baseline on unseen data in absolute terms (R^2 of 0.745 vs. 0.571).

Figure 5.5 shows a graphical representation of the baseline and PANN generalization gap evolution across training scenarios.

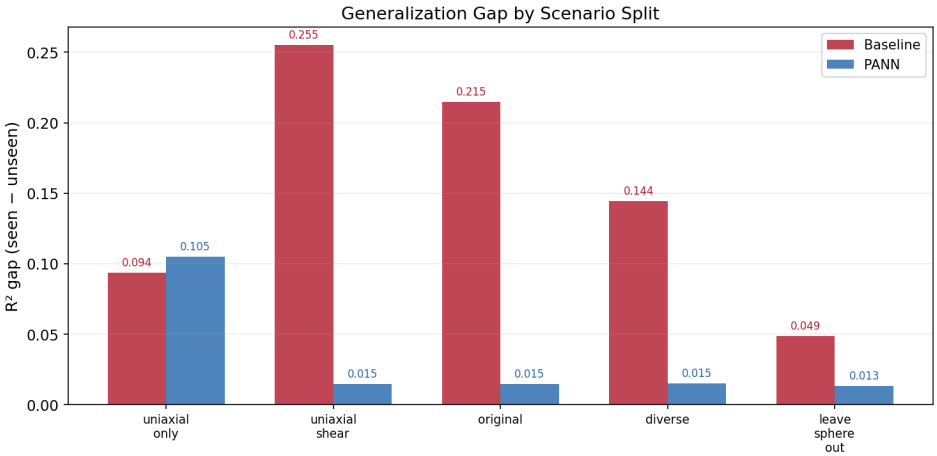

Figure 5.5: Generalization gap (R^2 seen minus unseen) by scenario split. The baseline gap varies from 0.049 to 0.255 depending on training diversity; the PANN gap is approximately constant at 0.013–0.015 after the first outlier of 0.105.

5.4.3 Per-Component Behavior in Unseen Scenarios

Table 5.9 breaks down the unseen R^2 by stress component for three representative splits.

Table 5.9: Per-component R^2 on unseen scenarios for three representative splits.

Split	Model	S_{11}	S_{22}	S_{12}
Uniaxial only	Baseline	0.923	0.803	−0.014
Uniaxial only	PANN	0.726	0.733	0.775
Original (7 scen.)	Baseline	0.793	0.740	0.652
Original (7 scen.)	PANN	0.990	0.988	0.997
Leave-sphere-out	Baseline	0.967	0.960	0.895
Leave-sphere-out	PANN	0.967	0.960	0.982

The per-component pattern is very consistent with the results obtained in Section 5.3, with S_{12} being the weakest component of the prediction in unseen scenarios (in the uniaxial-only split, the baseline's shear R^2 is -0.014, worse than predicting the mean, while the normal components remain in 0.923 and 0.803, respectively). In the PANN, again, the three components degrade together, as expected from the coupled learning mechanism described in Section 5.3.

The baseline shear prediction improves when shear-rich scenarios are included in the training set, confirming that shear predictions depend on having seen relevant examples during training. As a relevant example, in the uniaxial-only split the baseline's shear R^2 (-0.014) reflects that all the scenarios in the training set produced zero shear, so the baseline learns to predict zero for all inputs. The PANN captures the shear–normal coupling throughout the energy differentiation, making it far less dependent on the training composition. Figure 5.6 shows these two observations graphically.

5.5 Data Efficiency and Minimum Sample

5.5.1 Baseline Performance vs Training Size

In Section 5.4, we focused on understanding how varying training diversity affects performance on unseen scenarios and the generalization gap. In this section, we will

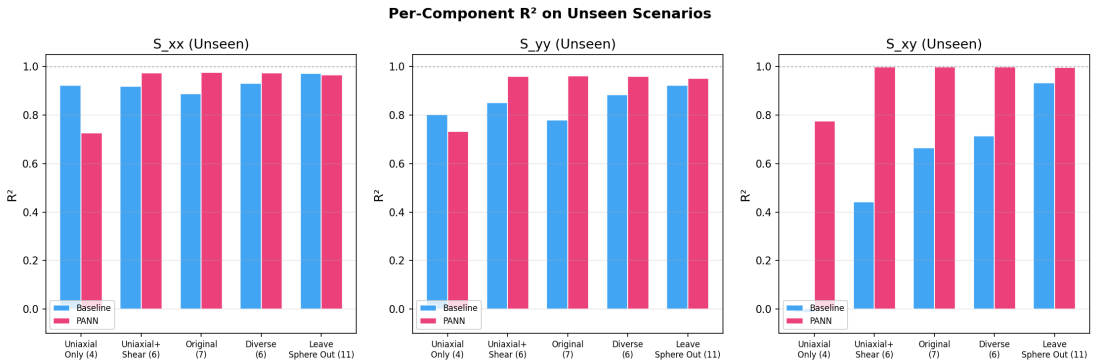

Figure 5.6: Per-component R^2 on unseen data for three representative splits. The baseline's S_{12} is consistently its weakest component; the PANN shows no such asymmetry.

analyze how much data each model needs from the same set of scenarios. We train both models on 10%, 25%, 50%, 75%, and 100% of the original split (Section 5.3) and track performance on both seen and unseen scenarios.

Table 5.10 reports the baseline's performance as a function of training set size.

Table 5.10: Baseline performance vs. training set size. Unseen R^2 peaks at 1,312 samples and declines with additional data.

Training samples	Seen R^2	Unseen R^2	Epochs (early-stopped)
262 (10%)	0.982	0.663	103
656 (25%)	0.991	0.825	102
1,312 (50%)	0.994	0.852	99
1,968 (75%)	0.993	0.823	82
2,625 (100%)	0.993	0.720	69

The baseline reaches $R^2 = 0.982$ on seen data with only a 10% sample and is essentially flat from 25% of the dataset (656 samples) onwards (interpolation is achieved early). On the other hand, the unseen R^2 peaks at 50% (1,312 samples) and then decreases as more data is added, dropping to 0.720 at full training size. This non-monotonic behavior in unseen scenarios, while maintaining consistently high performance in seen scenarios, is a typical signal for distribution-specific specialization (with more data from the same scenario types, the model specializes further to the training distribution and generalizes less to out-of-distribution loading

modes). The decreasing early-stopping epoch count reflects this same interpretation of the model converging to a distribution-specific solution.

5.5.2 PANN Performance vs Training Size

Table 5.11 reports the PANN's performance over the same data sizes.

Table 5.11: PANN performance vs. training set size. The model fails to converge below approximately 1,300 samples, then transitions sharply to high accuracy.

Training samples	**Seen R^2**	**Unseen R^2**	**Epochs**
262 (10%)	−0.098	−0.029	150
656 (25%)	0.041	0.092	150
1,312 (50%)	0.599	0.588	150
1,968 (75%)	0.989	0.965	150
2,625 (100%)	0.994	0.980	150

The PANN's data-efficiency profile has a very different interpretation compared to the baseline profile. In this case, the model fails to converge under 50% (1,300 samples), with negative R^2 at 10% (262 samples) and near zero at 656 (25%). At 50% (1,312 samples), the model is only partially fitted ($R^2 \cong 0.6$), and from this point the transition is steep, with R^2 values jumping over 0.9. This phased transition shows that, below a critical threshold (roughly 1,500 samples), the PANN cannot learn its energy function; above it, the model converges to high accuracy on both seen and unseen data without any signs of overfitting.

Figure 5.7 shows a graphical representation of the baseline and the PANN data efficiency profiles.

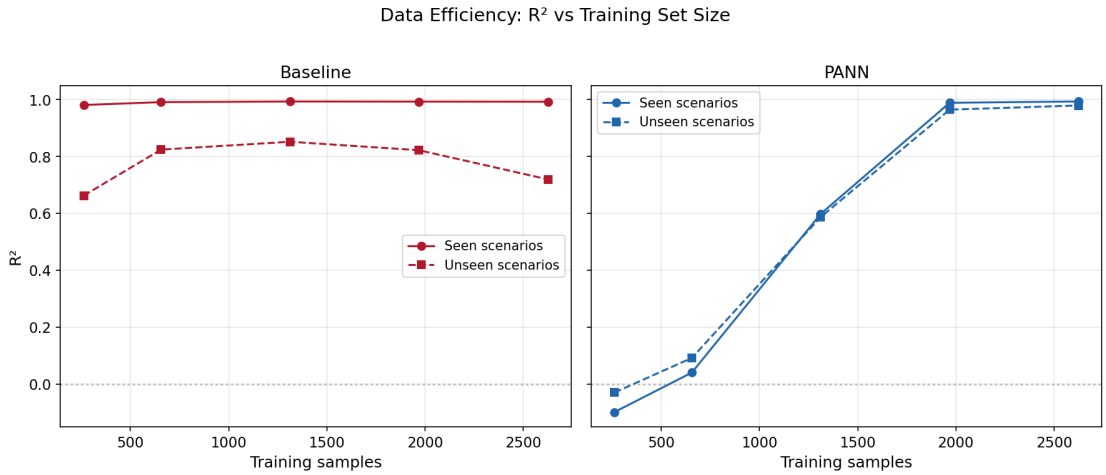

Figure 5.7: R^2 versus training set size for the baseline (left) and PANN (right). The baseline achieves high seen-scenario R^2 with very little data but its unseen R^2 peaks and then declines. The PANN fails completely below ∼1,300 samples, then rapidly converges to high accuracy on both seen and unseen data.

5.6 Training Dynamics

5.6.1 Early Stopping and Convergence

Figure 5.8 shows the training curves in their respective spaces (MSE Loss scaled for the baseline, MSE for the PANN) for the Original Split.

As introduced in Table 5.10, on the original split, the baseline trains for approximately 69 epochs before early stopping triggers. Its validation loss drops rapidly in the first 12 epochs, and then the training process continues, gradually reducing it. The gap between training and validation indicates overfitting, as discussed in Section 5.5.1.

For the PANN, the raw MSE loss decreases from 1.71×10^{11} to 1.76×10^{9} (roughly two orders of magnitude), with the validation loss following a very similar trajectory (no overfitting). In the PANN, early stopping does not trigger within the 150-epoch budget (Table 5.10), suggesting the model would benefit from longer training, although the curve shows minimal improvement in the final stage.

Figure 5.9 shows the PANN convergence for the different data fractions. Ex-

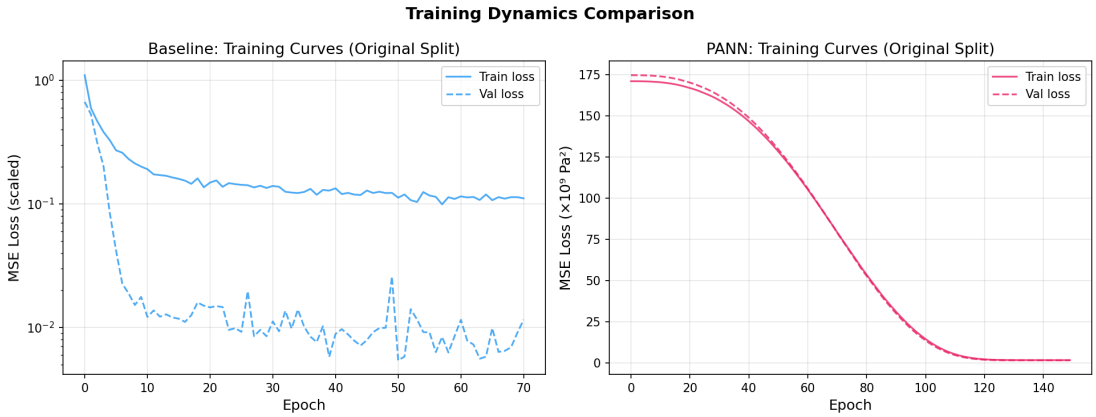

Figure 5.8: MSE loss for both models on the "original" split. The baseline (red) converges quickly and early-stops. The PANN (blue) shows a long plateau followed by rapid descent, and is still improving at epoch 150.

amining the training curves, the PANN's validation loss continues to decrease at epoch 150 in most runs. This means the results reported here likely underestimate the PANN's potential—a budget of 300 or 500 epochs would yield lower errors.

5.6.2 Training Time

Table 5.12 reports training wall time and epoch counts as a function of training set size.

Table 5.12: Training cost for both models across training set sizes. The PANN runs the full 150 epochs in every configuration; the baseline early-stops between 69 and 103 epochs.

	Baseline		**PANN**	
Samples	Time	Epochs	Time	Epochs
262	3.5 s	103	4.9 s	150
656	4.0 s	102	5.6 s	150
1,312	4.4 s	99	6.0 s	150
1,968	4.7 s	82	7.6 s	150
2,625	4.6 s	69	8.7 s	150

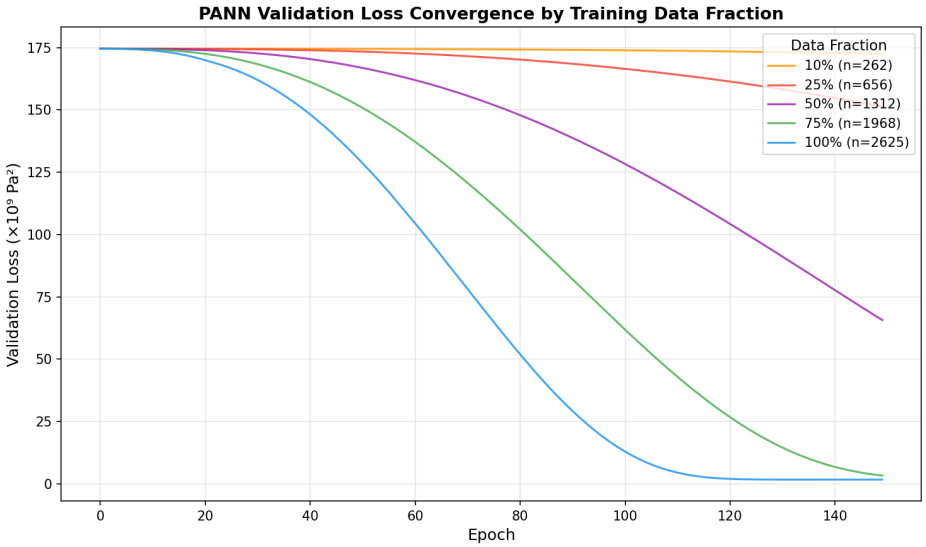

Figure 5.9: PANN validation loss across training epochs for five training set sizes. Larger datasets produce shorter plateau phases and faster convergence. At 10% (262 samples) and 25% (656 samples), the model remains on the plateau throughout training. At 50% (1,312 samples), convergence begins but is not complete by epoch 150. At 75% and 100% (1,968 and 2,625 samples), the model transitions into the rapid-descent phase and approaches convergence, with the full dataset converging earliest (around epoch 80).

As described in the previous section, the baseline's epoch count decreases with more data, consistent with the model converging faster to a narrower optimum as the loss landscape becomes more constrained. The PANN never triggers early stopping in any configuration, running all 150 epochs in every case. The per-epoch cost is similar between models (approximately 0.04–0.06 s per epoch). The PANN's total training time is 1.5–2 times the baseline's, driven by the higher epoch count rather than by computational overhead per step. This similarity in per-epoch cost, despite the (much) smaller size of the PANN relative to the baseline, stems from automatic differentiation via the energy function at each forward pass, which is computationally more expensive than a standard forward pass through dense layers.

In absolute terms, both models train in under 10 seconds on this dataset, so the cost difference is negligible for this problem size.

Figure 5.10 shows the overlap between epochs and wall time in both models.

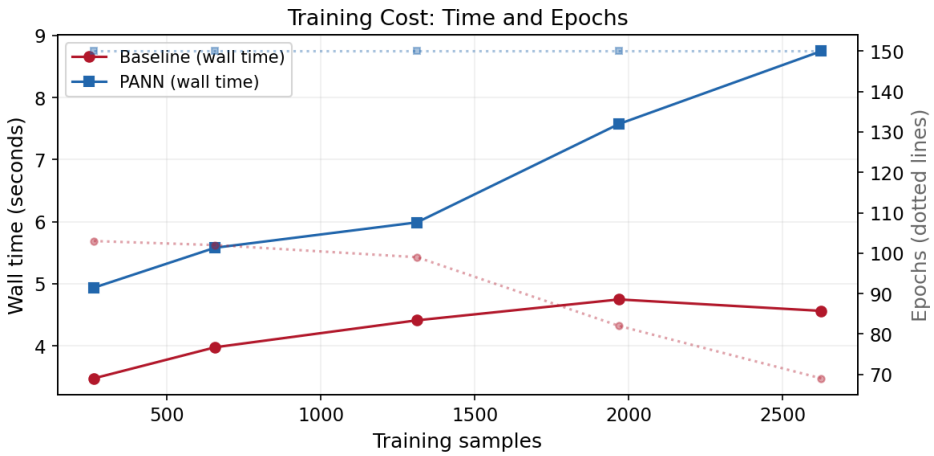

Figure 5.10: Training wall time (solid lines) and epoch count (dotted lines) versus training set size. The baseline early-stops in fewer epochs as data increases; the PANN always runs the full 150 epochs.

5.7 Discussion

The numerical results presented in this chapter allow us to address RQ2: enforcing physical constraints by construction yields a model that is more accurate and dramatically better at extrapolation. However, the results on data efficiency and training dynamics show that the two architectures have fundamentally different profiles that require further interpretation.

5.7.1 On Interpolation

Both models achieve R^2 above 0.99 on seen deformation modes, confirming that the constitutive map for a Neo-Hookean material under plane strain can be learned by either architecture if the training set covers the target loading modes.

The performance of the PANN is superior (39.2% lower RMSE and 53.5% lower MAE), but not by a margin that would, on its own, justify the effort of building it. In practice, however, no training dataset, no matter how carefully designed, can cover every possible strain state that a solver might encounter, and in engineering design we must consider the possibility of reaching regions of strain space that were never anticipated.

5.7.2 On Extrapolation

The generalization gap described in Section 5.3.4 is the most important result in this chapter. The baseline's R^2 drops by 26.6 percentage points when we move from seen to unseen deformation modes, while the PANN's R^2 drops by only 0.6 percentage points. Unlike the results observed on interpolation, this is a qualitative difference in performance.

The underlying mechanism explaining this behavior is overfitting: the baseline model has memorized a mapping that works well within the training distribution but has no mechanism to constrain its predictions outside it. The PANN, by contrast, is constrained to produce stress fields that are gradients of a convex potential, which limits the space of functions it can represent to those that are

physically plausible. This constraint acts as a guardrail during extrapolation, keeping predictions within the envelope of admissible hyperelastic responses.

The shear component, S_{12}, shows the largest performance drop for the baseline model, with R^2 dropping from 0.9973 to 0.6523 (34.5 percentage points and an RMSE ratio of 23x) on unseen data. This behavior is explained by the baseline learning three independent outputs, so its shear prediction is only as good as the shear signal in the training data. The PANN, in contrast, learns a single scalar energy from which all three stress components are derived by differentiation, so even when the training data contains limited shear information, the shear prediction benefits from the normal-stress training signal. As a result, all three components of the PANN's predicted stress degrade by similar amounts (RMSE ratios of 4× to 5×).

5.7.3 On Data Efficiency

The data-efficiency profiles of the two models are qualitatively different. The baseline learns quickly, reaching $R^2 > 0.98$ on seen data with just 262 samples (10% of the training set) and staying essentially flat from 656 samples onwards. The PANN, on the other hand, fails entirely below approximately 1,300 samples, producing negative R^2 values on both seen and unseen scenarios, but converges rapidly to an accurate solution above this threshold.

As discussed earlier, the baseline shows clear signs of overfitting, with unseen R^2 actually decreasing with more training data (peaking at 50% and declining to 0.72 at 100%), because additional samples from the same loading modes encourage the model to specialize further to the training distribution. In the PANN, there is no sign of the over-specialization that affects the baseline, and the unseen R^2 continues to increase even at high fractions of the training data.

This suggests that the PANN has a minimum "data budget" related to its training dynamics, which we explore further in the next section.

5.7.4 On Training Dynamics

As Table 5.12 shows, the PANN consistently runs all 150 training epochs without triggering early stopping, and the validation loss is still decreasing at the end of training in most configurations (see Figure 5.5). This means the results reported in this chapter likely underestimate the PANN's achievable accuracy, and a longer training budget (300–500 epochs) would yield lower errors, particularly for the smaller data fractions where the model is still in the plateau phase at epoch 150.

The training curves in Figure 5.5 also show that every run of the PANN sits on a loss plateau for a variable number of epochs and then either drops sharply or never drops at all. With 2,625 samples, the plateau breaks around epoch 40, while with 262 samples it does not break within the 150-epoch budget. This behavior is not present in the baseline, and it is a direct consequence of the physical constraints in the PANN.

In the baseline, the optimizer works with a direct relationship between weight adjustments and loss reduction. Any gradient step that reduces the loss is a valid step, and there are no restrictions on what values the weights can take.

The PANN optimizer faces a harder problem than its baseline counterpart, and the difficulty is a direct price of the physical guarantees. The optimizer cannot adjust the stress predictions directly, because the stress is not a learnable output (it is a derivative of the energy surface). In addition, after every gradient update, any weight that has gone negative is clipped to zero to satisfy the ICNN's non-negativity requirement. A large fraction of the computed gradient is discarded at each step because the descent direction the optimizer wants to take would violate the constraint. The effective step is only the component of the gradient that keeps all weights non-negative, which can be much smaller, or point in a very different direction, than the full gradient. Finally, the learnable scale parameter λ multiplies the entire corrected energy, creating a coupling between the scale and the shape of the energy function that the optimizer must resolve before meaningful progress can be made.

These three factors explain the plateau at low data fractions. With few samples, the gradient is noisy, and the fraction that survives the non-negative projection

points in an inconsistent direction from step to step. With more samples, the gradient signal is cleaner and the optimizer finds a consistent descent direction. Once the descent starts, convergence is fast because the space of admissible functions is smaller than in the unconstrained case — once the optimizer enters the right region, there are few wrong directions left.

The 1,300-sample threshold is not a fixed property of the PANN architecture: it depends on the ICNN size, the optimizer, and the initialization values.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This bachelor's thesis aimed to answer two questions. The first (RQ1) asked what strategies exist for enforcing physical constraints in machine learning-based constitutive models. The second (RQ2) asked whether enforcing those constraints by construction in the neural network architecture yields measurable benefits over a naive black-box approach in data efficiency and extrapolation.

To answer RQ1, Chapter 3 presented a structured review organized around three waves of development: early black-box models (1990s–2000s), deeper networks enabled by modern frameworks (2010s), and physics-constrained architectures (2020s). We proposed a taxonomy distinguishing soft constraints (loss-level penalties), hard constraints (architectural enforcement), and unsupervised methods (learning without stress labels). This taxonomy is not exhaustive but provides a framework we hope will be useful for researchers entering the field.

To answer RQ2, Chapters 4 and 5 described the implementation and evaluation of two models: a conventional feedforward baseline and a Physics-Augmented Neural Network (PANN) built on an Input-Convex Neural Network (ICNN) with energy corrections. Both were trained on synthetic stress–strain data generated using the Neo-Hookean law in Kratos Multiphysics under plane-strain conditions. The use of synthetic data removes experimental noise and geometric variability,

providing a controlled environment where performance differences can be attributed to the architecture rather than to data quality.

The numerical results support the main conclusion of this work: Physics-constrained architectures extrapolate better. On unseen deformation modes, the PANN maintained an R^2 above 0.99 while the baseline dropped to 0.73 — a degradation of 0.6 percentage points for the PANN versus 26.6 for the baseline. This gap was consistent across five different training splits with varying scenario composition. The baseline's generalization depended heavily on which loading modes were included in the training set, while the PANN's did not.

6.2 Contributions

This work makes the following contributions:

A structured literature review of the field of physics-integrated machine learning for hyperelastic constitutive modeling, organized as a historical progression across three waves and categorized by enforcement strategy (Chapter 3). To our knowledge, this is one of the few reviews that presents the field as an evolving response to specific technical limitations rather than as a flat catalogue of methods.

A complete, open-source implementation of both a baseline feedforward network and a PANN for isotropic hyperelastic materials under plane strain, including the synthetic data generation pipeline. Both repositories are publicly available under the MIT License, and the modular structure is designed so that individual components (the constitutive law backend, the network architecture, the evaluation pipeline) can be replaced independently.

A controlled experimental comparison that isolates the effect of architectural physics enforcement from confounding factors such as data noise, geometric variability, and material complexity. The use of a known analytical ground truth (Neo-Hookean) allows exact error quantification, which is not possible with experimental data.

6.3 Future Work

After this work, the most obvious next step is testing the PANN on harder problems. The Neo-Hookean model is smooth, well-behaved, and sits comfortably within the ICNN's representable class. Whether the same architecture holds up for Ogden or Mooney–Rivlin materials — where some energy terms are linear in the principal stretches and fall outside what invariant-based ICNNs can represent — is an open question. Testing on real experimental data, with noise and material variability, would be an even harder and more useful benchmark.

A second direction is embedding the PANN inside a finite element solver. This work stopped at pointwise stress evaluation, but the real motivation for polyconvex architectures is robust convergence of the Newton–Raphson iteration in nonlinear FE analysis. Implementing the PANN as a material subroutine in Kratos or Abaqus would test whether the theoretical stability guarantee translates into practical solver performance.

On the architecture side, two recent alternatives to the invariant-based ICNN deserve investigation: CSSV-NNs, which use signed singular values of the deformation gradient and can represent a broader class of polyconvex functions, and ICKANs, which replace dense layers with univariate splines and offer the possibility of extracting closed-form constitutive laws via symbolic regression.

Finally, extending the framework to history-dependent materials (plasticity, viscoelasticity, damage) is the natural long-term direction. This requires internal state variables and satisfaction of the Clausius–Duhem inequality, which introduces a different set of architectural constraints — but the core idea of replacing soft penalties with hard guarantees remains the same.

Appendices

Appendix A

Sustainability Analysis and Ethical Implications

A.1 Sustainability Analysis

A.1.1 Environmental Impact

Development of the bachelor's thesis

The environmental impact of this bachelor's thesis is dominated by the computational cost of training and evaluating the neural networks. The three main sources of energy consumption are (a) the process of synthetic data generation using Kratos Multiphysics, which involves evaluating a Neo-Hookean constitutive law at 15,925 strain states, (b) the training of the baseline and PANN models over multiple epochs using TensorFlow, and (c) the general energy cost of code development and preparation of the written document.

All work was carried out on a MacBook Air 13-inch with an Apple M4 chip (10-core CPU, fanless design), which consumes 3.63 W at idle[1]. While Apple does not report power consumption under sustained computational load, we estimate

[1] https://www.apple.com/environment/pdf/products/notebooks/M4_MacBook_Air_PER_March2025.pdf

a conservative average of 15 W over an estimated 300 hours of development, computation, and document preparation, resulting in a total energy consumption of approximately 4.5 kWh. Using the average carbon intensity of the Spanish electricity grid in 2025 published by the CNMC (0.283 kg CO2e/kWh)[2], the estimated carbon footprint of the computational work is approximately 1.27 kg CO2e. It is important to note that the server-side energy cost of AI-assisted tools such as Copilot is not included in this estimate, as the provider (Microsoft) does not publicly disclose the per-query energy consumption.

The local carbon footprint is approximately 1.27 kg CO2e, roughly equivalent to driving a new passenger car 12 km (106.8 g CO2/km)[3].

The project consumed no physical material or laboratory resources beyond computation. All data is synthetic, so there is no environmental cost of experimental testing (preparation of specimen, machine operations or waste disposal). The bachelor's thesis document itself was prepared digitally, with no printing required.

Beyond the energy consumption of the development of this bachelor's thesis, it is worth placing this energy cost in the context of the problem the work addresses. When we evaluate a constitutive law, such as Neo-Hookean, in a finite element simulation, the computational cost of the point operation is negligible. Training a neural network to reproduce the same law is, by comparison, much more expensive, and makes no sense from a pure energy standpoint. However, the real value of learned constitutive laws is not on the evaluation cycle, but on the process of finding the law. For materials where no adequate analytical form exists, a modeling engineer faces iterative cycles of experimental testing, model selection, and parameter calibration. If a physics-augmented neural network can learn a reliable constitutive law from a smaller experimental dataset, as the data efficiency results in Chapter 5 suggest, the net energy balance clearly favors the data-driven approach, as it replaces a far more expensive process upstream.

[2] https://canviclimatic.gencat.cat/en/actua/factors_demissio_associats_a_lenergia/

[3] https://www.eea.europa.eu/en/newsroom/news/average-co2-emissions-from-new-cars-and-new-vans

Risks and Limitations

The environmental analysis above is limited to the direct computational footprint of this specific implementation. If the methods developed here were scaled to larger models, three-dimensional problems or extensive hyperparameter searches, the energy consumption would scale. The growing computational demand of machine learning and deep learning model training is a recognized environmental concern for the field, thought the architectures explored in this work (small, physics-constrained networks) are necessarily more efficient than large-scale architectures.

A.1.2 Economic Impact

Development of the bachelor's thesis

The economic cost of this project can be estimated from two components: human resources and computational resources.

This bachelor's thesis corresponds to 12 ECTS in the study plan for the Civil Engineering Degree (2020), representing approximately 300 hours of work for the student. On top of that, we estimate 20-30 hours of supervision for each one of the three thesis advisors. We do not assign explicit hourly rates to these contributions, as the work has been done within an academic framework and all the cost is covered by the university structure.

As mentioned in A.1.1, the computational infrastructure used for this project is limited to a personal laptop (MacBook Air), with no additional hardware cost. All coding libraries used in the development of the project (Python, TensorFlow, Kratos Multiphysics, Numpy, scikit-learn and Matplotlib) are open-source and freely available. The only paid software tools were an Overleaf subscription (19€) for the preparation of the written document and a GitHub Copilot subscription (9€) for AI-assisted code development.

The two code repositories are published in GitHub under the MIT License, and the written document and results are available to other researchers at no cost.

Risks and Limitations

The economic limitations for this project are tied to the scope constraints imposed by the 12 ECTS format of the bachelor's thesis, that led us to restrict the number of architectures implemented and the depth of the study. A more comprehensive comparison (including, for instance, Kolmogorov–Arnold Networks or symbolic regression methods) would require a significant amount of time and potentially access to much more powerful computing resources. For the development of the project, we decided to rely on synthetic data to have a controlled ground-truth for evaluation. An experimental validation would also represent a relevant cost that must be considered when extending this research line.

A.1.3 Social Impact

Development of the bachelor's thesis

One of our explicit goals when developing this bachelor's thesis was to provide a solid foundation for future students and researchers to continue advancing the field of physics-integrated machine learning for constitutive modeling. Specifically, in Chapters 2 and 3, we made a deliberate effort to provide a comprehensive background for others to follow the work, including an extensive literature review and a structured taxonomy designed to serve as a reference for newcomers to the field. All the code has been published under the MIT License so that others can reproduce, modify, and extend the implementations without barriers.

The written document makes no use of gendered language or stereotypes, and inclusive language has been used throughout.

Risks and Limitations

The main social risk associated with this line of research is the potential for misuse of data-driven constitutive models, especially when working with black-box models or large-scale networks. If these techniques are not properly managed (using an adequate evaluation framework), they can potentially lead to unsafe structural

predictions. This risk is partially mitigated by the physics-augmented approach, which guarantees certain physical constraints by construction, but it does not eliminate the need for rigorous validation before any practical application.

A.2 Ethical Implications

This bachelor's thesis aims to demonstrate that physics-constrained architectures provide a more reliable alternative to black-box data-driven models in constitutive modeling, with clear advantages in data efficiency and extrapolation capabilities that can help avoid the risk of overconfidence. From this perspective, advancing the data-driven constitutive modeling field towards a more transparent and reliable space is a contribution to a more ethical engineering.

On the other hand, all data used has been generated synthetically, without involving human subjects, sensitive information, or proprietary data, and this work raises no concerns related to data privacy, consent, or intellectual property.

All third-party software and references are properly cited and acknowledged.

A.3 Relation to the Sustainable Development Goals

This thesis contributes to **SDG 9 (Industry, Innovation and Infrastructure)**, specifically target 9.5 ("Enhance scientific research, upgrade the technological capabilities of industrial sectors"), by developing and openly publishing tools that advance the integration of machine learning into structural engineering practice.

It also relates indirectly to **SDG 4 (Quality Education)**, target 4.7, through the educational intent of the literature review (Chapter 3), which was designed as a structured resource for future students and researchers entering the field of physics-integrated machine learning for constitutive modeling.

Appendix B

Software Organization

As described in Section 4.1.3, code has been implemented in two separate repositories (synthetic data generation and model pipelines).

For the data generation repository (`neohookean-data-generator`), the structure is as follows:

```
neohookean-data-generator/
 Makefile                   Shortcut commands for common project tasks
 requirements.txt           Python package dependencies
 archive/
    original_script.py     Original monolithic script (baseline reference)
 config/
    experiment.yaml        Central experiment parameters and settings
 data/
    metadata.json          Dataset metadata summary
 notebooks/                  Jupyter notebooks for analysis
 scripts/
    consolidate_data.py    Merges outputs into unified datasets
    generate_data.py       Entry script for synthetic data generation
 src/
     neohookean_generator/
         __init__.py         Package API
         config.py           Configuration loading and validation
```

```
physics.py          Neo-Hookean material computations
sphere_generator.py Sphere-based synthetic sample generation
visualization.py    Plotting helpers for generated data
```

For the model pipelines repository (`neohookean-ml-pipeline`), the structure is as follows:

```
neohookean-ml-pipeline/
 Makefile                   Build/run workflow shortcuts
 requirements.txt           Python dependencies
 main.py                    Main entry point for the pipeline
 test_case_splitting.py     Dataset split validation script
 configs/
    default.json           Default pipeline configuration
    physics.json           Physics-focused configuration overrides
 notebooks/                 Experiments, diagnostics, and analysis
 results/
    baseline/
       history.json       Training history (loss/metrics per epoch)
       seen_metrics.json  In-distribution evaluation metrics
       unseen_metrics.json Out-of-distribution evaluation metrics
    pann/
        history.json       Training history (loss/metrics per epoch)
        seen_metrics.json  In-distribution evaluation metrics
        unseen_metrics.json Out-of-distribution evaluation metrics
 src/
     __init__.py            Package module root
     comparison.py          Model/run comparison utilities
     data.py                High-level data handling helpers
     visualization.py       Plotting helpers for metrics/results
     data/
        __init__.py
        loader.py           Data loading into model-ready structures
        preprocessor.py     Feature/target preprocessing
     models/
        __init__.py
        base_model.py       Baseline model implementation
        network.py          Neural network architecture components
        pann_model.py       PANN model implementation
```

```
utils/
    __init__.py
    evaluation.py     Metric computation and evaluation helpers
```

Appendix C

Dependencies

The implementation is built on the following core libraries:

- **TensorFlow** ($\geq$ 2.8.0): The deep learning framework used for both model architectures. TensorFlow's `keras` API defines the model layers and training loops.
- **NumPy** ($\geq$ 1.21.0): Numerical array operations throughout data loading, preprocessing, and metric computation.
- **scikit-learn** ($\geq$ 1.0.0): Provides `train_test_split` for data splitting, standard scaler for input/output normalization (baseline model only), and regression metrics (`r2_score`, `mean_squared_error`, `mean_absolute_error`).
- **Matplotlib** ($\geq$ 3.4.0): All visualization utilities, including training curves, prediction scatter plots, residual analysis, and comparison charts.
- **pandas** ($\geq$ 1.3.0): Used in the comparison module to generate summary DataFrames for tabular metric reporting.
- **Jupyter** ($\geq$ 1.0.0): Notebook environment for interactive training, evaluation, and visualization.

In the data generation pipeline, the core dependency is **KratosMultiphysics**.

Additional dependencies include NumPy, Matplotlib, and PyYAML, which are widely known Python libraries.

References

Abadi, Martín, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng (2016). "TensorFlow: A System for Large-Scale Machine Learning". In: *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*. Savannah, GA: USENIX Association, pp. 265–283.

Amos, Brandon, Lei Xu, and J. Zico Kolter (2017). "Input Convex Neural Networks". In: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. Vol. 70. PMLR. Sydney, Australia, pp. 146–155.

Ball, John M. (1977). "Convexity Conditions and Existence Theorems in Nonlinear Elasticity". In: *Archive for Rational Mechanics and Analysis* 63.4, pp. 337–403.

Bonet, Javier and Richard D. Wood (2008). *Nonlinear Continuum Mechanics for Finite Element Analysis*. 2nd. Cambridge, U.K.: Cambridge University Press.

Bourdyot, M., M. Compans, R. Langlois, B. Smaniotto, E. Baranger, and C. Jailin (2025). "Learning a Hyperelastic Constitutive Model from 3D Experimental Data". In: *Computer Methods in Applied Mechanics and Engineering*. In press.

Chen, Peiyi and Johann Guilleminot (2022). "Polyconvex Neural Networks for Hyperelastic Constitutive Models: A Rectification Approach". In: *Mechanics Research Communications* 125, p. 103993.

Chung, Isu, Seyoung Im, and Maenghyo Cho (2021). "A Neural Network Constitutive Model for Hyperelasticity Based on Molecular Dynamics Simulations". In: *International Journal for Numerical Methods in Engineering* 122.1, pp. 5–24.

Dadvand, Pooyan, Riccardo Rossi, and Eugenio Oñate (2010). "An Object-Oriented Environment for Developing Finite Element Codes for Multi-Disciplinary Applications". In: *Archives of Computational Methods in Engineering* 17.3, pp. 253–297.

Dean, Jeff (2020). "The Deep Learning Revolution and Its Implications for Computer Architecture and Chip Design". In: *Proceedings of the 2020 IEEE International Solid-State Circuits Conference (ISSCC)*. San Francisco, CA: IEEE, pp. 8–14.

Flaschel, Moritz, Siddhant Kumar, and Laura De Lorenzis (2021). "Unsupervised Discovery of Interpretable Hyperelastic Constitutive Laws". In: *Computer Methods in Applied Mechanics and Engineering* 381, p. 113852.

Fuhg, Jan N. and Nikolaos Bouklas (2022). "On Physics-Informed Data-Driven Isotropic and Anisotropic Constitutive Models through Probabilistic Machine Learning and Space-Filling Sampling". In: *Computer Methods in Applied Mechanics and Engineering* 394, p. 114915.

Furukawa, Tomonari and Genki Yagawa (1998). "Implicit Constitutive Modelling for Viscoplasticity Using Neural Networks". In: *International Journal for Numerical Methods in Engineering* 43.2, pp. 195–219.

Geuken, Gian-Luca, Patrick Kurzeja, David Wiedemann, and Jörn Mosler (2025). "A Novel Neural Network for Isotropic Polyconvex Hyperelasticity Satisfying the Universal Approximation Theorem". In: *Journal of the Mechanics and Physics of Solids*. June 2025.

Ghaboussi, Jamshid, Jr. Garrett James H., and Xiping Wu (1991). "Knowledge-Based Modeling of Material Behavior with Neural Networks". In: *Journal of Engineering Mechanics* 117.1, pp. 132–153.

Ghaboussi, Jamshid, David A. Pecknold, Mingfu Zhang, and Rami M. Haj-Ali (1998). "Autoprogressive Training of Neural Network Constitutive Models". In: *International Journal for Numerical Methods in Engineering* 42.1, pp. 105–126.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. Cambridge, MA: MIT Press.

Hornik, Kurt (1991). "Approximation Capabilities of Multilayer Feedforward Networks". In: *Neural Networks* 4.2, pp. 251–257.

Jung, Sungmoon and Jamshid Ghaboussi (2006). "Neural Network Constitutive Model for Rate-Dependent Materials". In: *Computers & Structures* 84.15–16, pp. 955–963.

Kingma, Diederik P. and Jimmy Ba (2015). "Adam: A Method for Stochastic Optimization". In: *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. San Diego, CA.

Koeppe, Arnd, Franz Bamer, Michael Selzer, Britta Nestler, and Bernd Markert (2022). "Explainable Artificial Intelligence for Mechanics: Physics-Explaining Neural Networks for Constitutive Models". In: *Frontiers in Materials* 8, p. 824958.

Linden, Lennart, Dominik K. Klein, Karl A. Kalina, Jörg Brummund, Oliver Weeger, and Markus Kästner (2023). "Neural Networks Meet Hyperelasticity: A Guide to Enforcing Physics". In: *Journal of the Mechanics and Physics of Solids* 179, p. 105363.

Linka, Kevin, Markus Hillgärtner, Kian P. Abdolazizi, Roland C. Aydin, Mikhail Itskov, and Christian J. Cyron (2021). "Constitutive Artificial Neural Networks: A Fast and General Approach to Predictive Data-Driven Constitutive Modeling by Deep Learning". In: *Journal of Computational Physics* 429, p. 110010.

Liu, Wing Kam, Shaofan Li, and Harold S. Park (2022). "Eighty Years of the Finite Element Method: Birth, Evolution, and Future". In: *Archives of Computational Methods in Engineering* 29, pp. 4431–4453.

Liu, Zeliang, C. T. Wu, and Masataka Koishi (2019). “A Deep Material Network with Cohesive Layers: Multi-Stage Training and Interfacial Failure Analysis”. In: *Journal of the Mechanics and Physics of Solids* 129, pp. 20–46.

Maslej, Nestor, Loredana Fattorini, Raymond Perrault, Yolanda Gil, Vanessa Parli, Njenga Kariuki, Emily Capstick, Anka Reuel, Erik Brynjolfsson, John Etchemendy, Katrina Ligett, Terah Lyons, James Manyika, Juan Carlos Niebles, Yoav Shoham, Russell Wald, Toby Walsh, Armin Hamrah, Luca Santarlasci, João B. Lotufo, Anirudh Rome, Alex Shi, and Sukrut Oak (2025). *The AI Index 2025 Annual Report*. Tech. rep. Accessed April 10, 2026. Stanford, CA: AI Index Steering Committee, Institute for Human-Centered AI, Stanford University. URL: https://hai.stanford.edu/ai-index/2025-ai-index-report.

Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala (2019). “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Proceedings of the 33rd International Conference on Neural Information Processing Systems (NeurIPS)*. Vancouver, Canada: Curran Associates, pp. 8024–8035.

Raissi, Maziar, Paris Perdikaris, and George Em Karniadakis (2019). “Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations”. In: *Journal of Computational Physics* 378, pp. 686–707.

Shen, Yiping, K. Chandrashekhara, William F. Breig, and L. R. Oliver (2005). “Finite Element Analysis of V-Ribbed Belts Using Neural Network Based Hyperelastic Material Model”. In: *International Journal of Non-Linear Mechanics* 40.6, pp. 875–890.

Thakolkaran, Prakash, Yuezhe Guo, Suraj Saini, Mathias Peirlinck, B. Alheit, and Siddhant Kumar (2025). “Can KAN CANs? Input-Convex Kolmogorov–Arnold

Networks (KANs) as Hyperelastic Constitutive Artificial Neural Networks (CANs)". In: *Computer Methods in Applied Mechanics and Engineering* 443, p. 118089.

Thakolkaran, Prakash, Akash Joshi, Yiwen Zheng, Moritz Flaschel, Laura De Lorenzis, and Siddhant Kumar (2022). "NN-EUCLID: Deep-Learning Hyperelasticity without Stress Data". In: *Journal of the Mechanics and Physics of Solids* 169, p. 105076.

Theocaris, Pericles S. and Panagiotis D. Panagiotopoulos (1995). "Neural Networks for Computing in Fracture Mechanics. Methods and Prospects of Applications". In: *Computer Methods in Applied Mechanics and Engineering* 125.1–4, pp. 191–208.